

Final project Database design and development

1. Introduction

Scenario (Introduction)

Jasmine Restaurant is aiming to develop and overcome the issues that the restaurant has been suffering from. Relying on paper records which can be easily destroyed and ruined or even get lost, and on spreadsheets which lack organization makes it difficult for the restaurant management to track the restaurant income and expenses or being able to determine the weakness that may affect the restaurant's reputation if they remain unresolved.

Delivery Management

The restaurant offers a delivery service that allows customers users to place orders from home and have them delivered to the doorstep. Because of that it is very important to organize such operation professionally. Making a strong delivery management system will help the restaurant to increase profitability, expand business operations, and enhance customer satisfaction. To make such a system it should contain information about the customer, driver, order, payment record and delivery status.

The objective of this technical documentation is to come up with a systematic method of designing, implementing and maintaining the relational database of Jasmine Restaurant. This can be achieved through technical documentation that provides integrity, security and scalability. The document will describe the need of the user, system and data. After this, there will be a description of the database design that is conceptual, logical and physical. In addition, this document will cover SQL development, security and privilege, maintenance and backup, testing and validation and technical evaluation of the database.

2 Database Requirement

(functional requirements):

Customer requirements:

1. Sign up: full name, username, phone number, email, date of birth, password, gender, Addresses.
2. Sign in: username and password
3. Forgot password button
4. Search box: to search for different dishes and items
5. Filter: which filters the search results depending on user decision depending on price ingredients, time it needs to get ready, food type (sweet, main meal dish, vegetarian dish, juice or snacks)
6. Order Basket contains all the dishes that the customer adds to order
7. Rating restaurant: from five stars (5 stars is the best while 0 star is the worst) including a comments section that customers can express their opinions at
8. Tracking order: showing the predictable time that the customer will get the order in, and in which step the order is in (Preparation phase, ready waiting to be delivered phase, Delivery phase)
9. Restaurant information (bio): its location its food specialty (fast food, Italian food, etc.) and Date of establishment

10. Dish information: price, size and ingredients
11. Discount section: the amount of discount and on what dishes, how long is the discount valid? and the price before and after the discount

Driver requirements:

1. Sign in: username and password
2. Forgot password button
3. Viewing any notification from the restaurant
4. Tracking order: showing the predictable time that the customer will get the order in, and the distance between the customer and the restaurant.

Admin requirements:

1. Sign in: username and password
2. Forgot password button
3. Sending alerts and warnings to drivers in the event of delays or violations of any work rules.
4. Manage restaurant profile and operational information.
5. Restaurant information, seeing different private information about the restaurant ex (profit, cost)
6. Manage Discounts and Pricing
7. Assigning drivers to orders: Allocating new orders to available drivers.

Non-Functional Requirements (System Requirements):

- 1) Fast: going from page to another in less than 1 second
- 2) Secure: having a proxy firewall, VPN and all data should be encrypted
- 3) Well determined privilege: each employee should have the least privilege allows them to perform their assigned duties
- 4) Easy to use: customers don't find any difficulty while using the app
- 5) Good capacity: being able to serve 1000 customers at the same time at the app
- 6) Good scalability: each year the app should be able to support an annual user growth rate of 25%
- 7) Backup: save the data in a backup every day at 1:00 am
- 8) Easy to update: the developers can add extra features easily
- 9) Available: customers can use the app every day at any time (24/7 system availability).

2.2 Data Requirements

- Customer:
The person who makes orders, one customer, can make one order, many orders or even zero orders. The system maintains records for registered customers who place delivery orders.

The system should have in its data set customer full name (composed of a first name and a last name), date of birth, phone number (should be unique) email address (should be unique), password, gender (optional) the gender can be 'female' or 'male' only. As customers have the requirement for deliveries at different places, a customer can also register several delivery addresses, wherein every address has the mention of the city, street, and type of address (Home/Work).

- Driver:

They are the restaurant employees whose job is to deliver the orders to the customer. Each driver can deliver one order, many orders or even zero orders (if he is absent that day).

The system should have in its data set Driver full name, date of birth, phone number (should be unique) email address (should be unique), password, gender (optional) the gender can be 'female' or 'male' only.

- Order:

A dish or a group of dishes on restaurant menu in the app which a customer ordered and the driver delivered. The restaurant offers a diverse range of food and beverage items on its menu. Each of these items has a distinct name, a unique category (main dish, side dish, drinks, and desserts), and a corresponding price per unit. Each order can have at least one item or many items, and each item can be added to one order or to many orders.

Every one order can be ordered by one customer and delivered by one driver and can also have one payment record and one delivery status

Each order will have the items it contains from the restaurant menu, their price and quantity. It should also have information about estimated time for the customer to receive the order.

- Payment record:

For every completed or placed order, the system maintains a corresponding financial payment record. For every order process will be one payment record and each payment record will be indicated to one order only and only one.

Each payment record will contain the method of payment (cash, credit card, CliQ) if the customer did not determine the payment method it will be considered as cash by default. The payment record should also contain the amount that has been paid by the customer and the specific date (year, month, month, day, hour, minute, second)

- Delivery Status:

Each order is linked to a real-time delivery status record that tracks its progression. It determines the Delivery Status of the order (preparing, out for delivery, delivered or cancelled) and the specific updated time (year, month, month, day, hour, minute, second) also if there are any notes it will be mentioned.

3.1 Conceptual Design

Definition:

“Conceptual database design is the process of constructing a high-level model of the information used in an enterprise, completely independent of all physical considerations such as the target Database Management System (DBMS), application programs, programming languages, or hardware platforms. It transforms the collected business and data requirements into a structured, human-understandable visual representation” (Connolly and Begg, 2015)

Goal of Conceptual Design:

The main goal is to determine and organize the entities, their attributes and the relationships between them accurately. It serves as Unified conceptual blueprints between developers and stakeholders that make communication much easier and clearer. It ensures that all rules and requirements have been covered before diving into on the technical process.

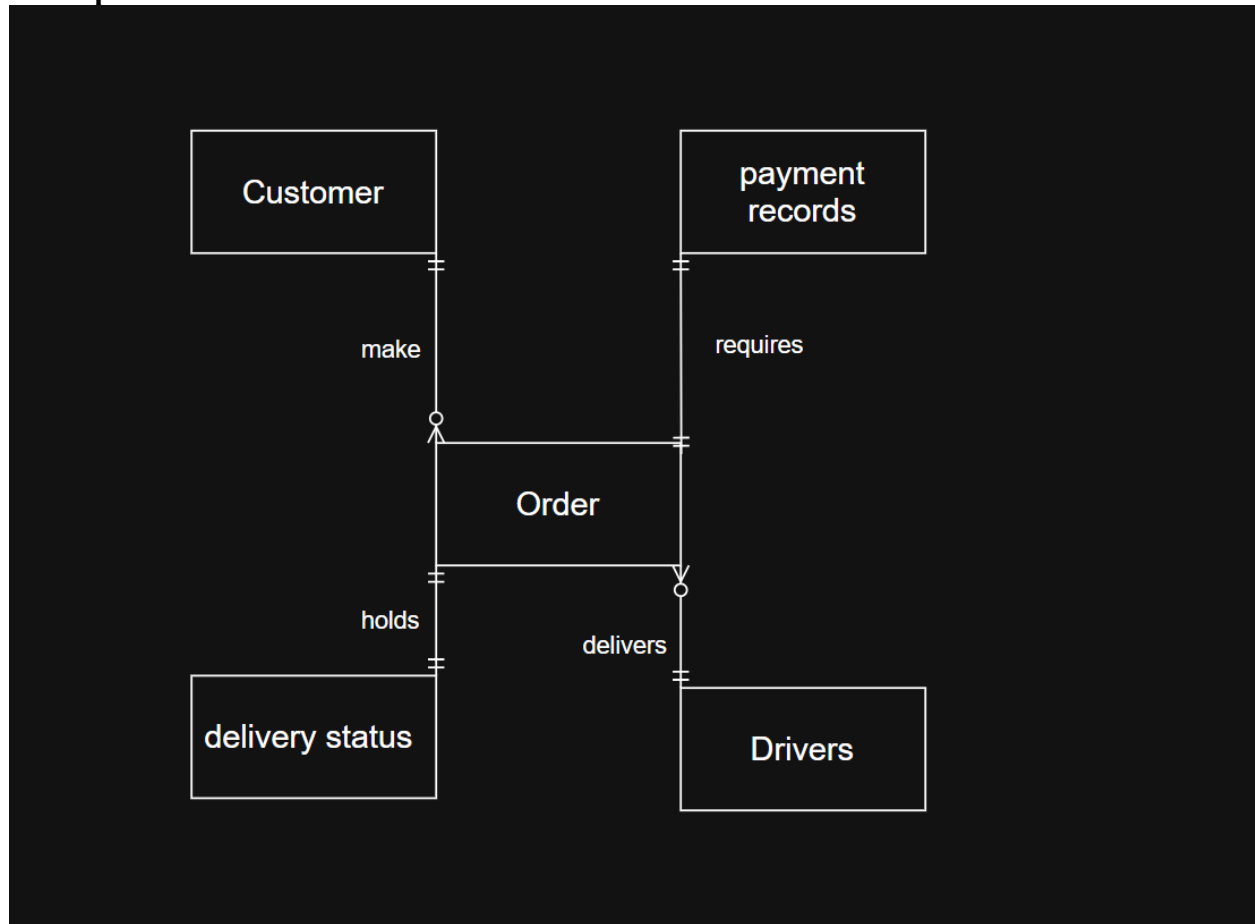
Components:

1. Entities: the objects, people or system concepts (Customer, Driver, Order...)
2. Attributes: The descriptive characteristics belonging to each entity, they have many types (simple, composite, multi-valued, and derived attributes).
3. Relationships: How different entities connect with each other.
4. Constraints: The rules that every value in the entity should be followed

The relationship between the entities:

- Customer: Order 0-M
- Order: Customer 1:1
- Drivers: Order 0-M
- Order: Drivers 1:1
- Order: delivery status (1:1)
- delivery status: Order (1-1)
- Order: payment records (1:1)
- payment records: Order 1:1

Conceptual Schema:



Description:

Conceptual schema diagram depicts the main entities and relations that are involved in Jasmine Restaurant Delivery Management System as well as the basic business relationships and cardinality among them:

1. Customer to Order ("makes" relationship)

One-to-Many (1:M) relationship, when a customer, who has a valid account in the system, may make zero, one, or several orders over time (0,M), while each order is made by one particular customer (1,1)

2. Drivers to Order ("drives" relationship)

One-to-Many (1:M) relationship, when a driver drives zero, one, or several orders within one shift period (0,M), while each order is driven by one driver only(1,1).

3. Order to Payment records ("requires" relationship)

One-to-One (1:1) relationship, when any completed order should have one unique payment record only (1,1), while each record belongs to one order(1,1).

4. Order to Delivery Status ("holds" relationship)

One-to-One (1:1) relationship, when any order holds one delivery status record that shows its actual delivery status (Preparing, Out for Delivery, Delivering) at the moment and each status record refers to one order.

Order entity acts as a center of the conceptual schema, connecting customers, drivers, payments and tracking records in an effective way.

3.2 Mapping: (First Normalization Form):

Definition:

- it is a set of clear rules and steps that should be followed in order to create a correct relationship between entities in the database system and also to create a logical schema from the conceptual schema.

First step in mapping is that each entity should represent a table and each attribute should be a column:

Customer attributes:

Customer
Customer ID (PK)
Full name (first name, last name)
Age
Date of Birth
phone number
email
password
gender
Addresses

Driver attributes:

Drivers
Driver ID (PK)
Full name (first name, last name)
Age
Date of Birth
phone number
email
password
gender

Order attributes:

Order
Order ID (PK)
Order contents
Order driver ID
Order customer ID
Predictbal time
Order price

Delivery Status attributes:

Delivery Status
Status ID (PK)
Order ID (FK)
Delivery Status name
Updated At / Status Time
Notes

Payment records attributes:

Payment records
Payment ID (PK)
Order ID(FK)

Payment Method (Cash, Credit Card, E-Wallet)
Amount Paid
Payment Date

Each entity now being represent as a table and each attribute as column, and the primary key (PK) and foreign key (FK)

Primary Key:

It is a specific column or set of columns in the table that uniquely identifies each row (record) and makes reaching its information much easier and faster

Foreign Key:

a column or group of columns in a table that connect the first table with the second table by adding the second table primary key as foreign key in the first table.

There are a lot of types for attributes and dealing with each type depends on its type:

Derived Attributes:

Two attributes represent the exact information, an attribute that can be extracted from other, such as Age and date of birth, from the date of birth attribute the age is derived. So, the age should be deleted.

The customer and driver tables after this step:

Customer
Customer ID (PK)
Full name (first name, last name)
Date of Birth
phone number
email
password
gender
Addresses
Drivers
Driver ID (PK)
Full name (first name, last name)
Date of Birth
phone number
email
password
gender

Composite attributes:

Which when the attribute contains more than simple attribute. Like the full name attribute in both employee and customer tables, it should be divided into two columns (two attributes) first name attribute and last name attribute.

The customer table after this step:

Customer	
Customer ID (PK)	
first name	
last name	
Date of Birth	
phone number	
email	
password	
gender	
Addresses	

The driver table after this step:

Drivers	
Driver ID (PK)	
first name	
last name	
Date of Birth	
phone number	
email	
password	
gender	

Multi-valued attributes:

Attributes that contain more than one value, in this case a new table should be implemented. Example, Order contents attribute in Order table, this attribute might contain multiple values to it need to have its own new table.

The new table:

Order items	
Item name (PK)	Item name
Order ID(PK)	Order ID
Quantity	Quantity

Item category (main dish, side dish, juice...)	
Unit price	

Order table after this step

Order
Order ID (PK)
Order driver ID (FK)
Order customer ID (FK)
Predicted time
Order price

Complex attributes:

Attributes that are both composite and multi valued, like the customer addresses, each customer may have more than one address, and each address contains city address and street address. A new table will be implemented to fix this problem.

The new table:

Customer addresses
Customer ID (FK)(PK)
City address
Street address
Address type (home, work...) (PK)
Phone number

The customer table after this step:

Customer
Customer ID (PK)
first name
last name
Date of Birth
email
password
gender

In 1:M relationships, we include the PK from the primary table as a FK in the related table, such as adding CustomerID and DriverID as FKs in the Order table.

In 1:1 relationships, we include the PK as a FK to link both tables, such as adding OrderID as a FK in both Payment_Records and Delivery_Status tables.

4 Normalization

4.1 1 st NF

Relations	Attributes	Violation description	Solution – Relations
Customer: (Customer ID, Full Name, Age, Date of Birth, Phone Number, Email, Password, Gender, Addresses)	Full Name	The full name attribute is a composite attribute that can be divide into two simple attributes (first name and second name)	Customer: (Customer ID, First Name, Last Name, Age, Date of Birth, Phone Number, Email, Password, Gender, Addresses)
Customer (Customer ID, First Name, Last Name, Age, Date of Birth, Phone Number, Email, Password, Gender, Addresses)	Age	Age is derived attribute whose value can be extracted from DoB attribute. So, it is unnecessary attribute that should be deleted	Customer (Customer ID, First Name, Last Name, Date of Birth, Phone Number, Email, Password, Gender, Addresses)
Customer (Customer ID, First Name, Last Name, Date of Birth, Phone Number, Email, Password, Gender, Addresses)	Addresses	The Addresses attribute is a multi-valued and complex attribute, each customer can have more than one address and each address contain street address and city address	Customer (Customer ID, First Name, Last Name, Date of Birth, Email, Password, Gender) Customer Addresses (Customer ID(FK)(PK), Address Type(PK), City Address, Street Address, Phone number)
Drivers (Driver ID, Full Name, Age, Date of Birth, Phone Number, Email, Password, Gender)	Full Name	The full name attribute is a composite attribute that can be divide into two simple attributes (first name and second name)	Drivers (Driver ID, First Name, Last Name, Date of Birth, Phone Number, Email, Password, Gender, Age)
Drivers (Driver ID, First Name, Last Name, Date of Birth, Phone Number, Email, Password, Gender, Age)	Age	Age is derived attribute whose value can be extracted from DoB attribute. So, it is unnecessary attribute that should be deleted	Drivers (Driver ID, First Name, Last Name, Date of Birth, Phone Number, Email, Password, Gender)

Order (Order ID, Order Customer ID*(FK), Order Driver ID(FK)*, Predictable Time, Order Price, Order Contents)	Order Contents	Order Contents, it is a multi-valued attribute that can represent more than one contents. So it should be separated into different table	Order (Order ID, Order Customer ID*(FK), Order Driver ID(FK), Predictable Time, Order Price) Order Items (Item Name(FK), Order ID(FK)*, Quantity, Item Category, Unit Price)
---	----------------	--	---

The tables after mapping process:

Customer
Customer ID (PK)
first name
last name
Date of Birth
email
password
gender

Customer addresses
Customer ID (FK)(PK)
City address
Street address
Address type (home, work...) (PK)
Phone number

Drivers
Driver ID (PK)
first name
last name
Date of Birth
phone number
email
password

gender

Order
Order ID (PK)
Order driver ID (FK)
Order customer ID (FK)
Predictbal time
Order price

Order items
Item name (PK)
Order ID(PK)
Quantity
Item category (main dish, side dish, juice...)
Unit price

Payment records
Payment ID(PK)
Order ID (FK)
Payment Method (Cash, Credit Card, CliQ)
Amount Paid
Payment Date

Delivery Status
Status ID (PK)
Order ID (FK)
Delivery Status name
Updated At / Status Time
Notes

By this the mapping and first normalization form (1NF) have been ended.

4.2 2 nd NF

Second Normalization Form:

The second normalization form requirements:

1. Satisfy 1NF requirements
2. There should be no partial dependency.

Partial dependency happens when an attribute or set of attributes depends on a part of a composite key rather than the entire key.

Relations	FDs	Violation description	Solution Relations
Order Items (Item Name, Order ID, Quantity, Item Category, Unit Price)	FD1: (Item Name, Order ID) → Quantity FD2: Item Name → Item Category, Unit Price	In this table there is a composite key (Item name and Order ID), the quantity attribute fully depends on the entire key while item category and unit price only depend on part of the key (Item name), this causes many problems to the data set such as data redundancy and update anomalies the data set.	Order Items (Item Name (FK), Order ID(FK), Quantity) Menu Items (Item Name, Item Category, Unit Price)
Customer Addresses (Customer ID, Address Type, City Address, Street Address, Customer Phone Number)	FD1: (Customer ID, Address Type) → City Address, Street Address FD2: Customer ID → Customer Phone Number	The table has a composite primary key (Customer ID, Address Type) Street address and city address are fully depend on whole primary key while the phone number just depend on partial of it (customer ID)	Customer (Customer ID, First Name, Last Name, Date of Birth, Phone Number, Email, Password, Gender) Customer Addresses (Customer ID(FK), Address Type, City Address, Street Address)

4.3 3rd NF

Third normalization form (3NF):

Its requirements:

1. Satisfies 2NF requirements
2. Remove any transitive functional dependencies

The functional dependencies is when attribute Z depends on attribute Y which in turn depends on an X key which creates an indirect transitive relationship.

$$X \rightarrow Y \rightarrow Z$$

If there is this table in this logical schema:

Orders and Drivers	
Order ID (PK)	
Predictbal time	
Order price	
Driver ID	
Driver first name	
Driver last name	
Driver phone number	

Relations	FDs	Violation description	Solution – Relations
Orders and Drivers (Order ID, Predictable Time, Order Price, Driver ID, Driver First Name, Driver Last Name, Driver Phone Number, Customer ID)	Order ID $\rightarrow$ Driver ID FD2: Driver ID $\rightarrow$ Driver First Name, Driver Last Name, Driver Phone Number	It is obvious transitive functional dependencies issue, because the each one of Driver first name, Driver last name, Driver phone number are dependent on Driver ID attributes which in turn depends on order ID primary key.	Order (Order ID, Order Driver ID(FK), Order Customer ID(FK), Predictable Time, Order Price) Drivers (Driver ID, First Name, Last Name, Date of Birth, Phone Number, Email, Password, Gender)

If there is this table in this logical schema

Orders and Customer	
Order ID (PK)	
Predictable time	
Order price	
Customer ID	
Customer first name	
customer last name	
Customer phone number	

Relations	FDs	Violation description	Solution – Relations
Orders and Customers (Order ID, Predictable Time,	FD1: Order ID $\rightarrow$ Customer ID	Here there is a clear transitive functional dependency. The	Order (Order ID, Order Customer ID(FK), Order Driver

Order Price, Customer ID, Customer First Name, Customer Last Name, Customer Phone Number, Customer Email)	FD2: Customer ID → Customer First Name, Customer Last Name, Customer Phone Number, Customer Email	primary key in this table is order ID however in order to reach the customer first name the customer ID should be known and in order to know the customer ID, order ID is needed this create a transitive dependency Order ID → customer ID → Customer first name ...	ID(FK), Predictable Time, Order Price) Customer (Customer ID, First Name, Last Name, Date of Birth, Phone Number, Email, Password, Gender)
---	--	---	---

The benefits of applying Normalization forms:

- Elimination of Data Redundancy: when data separates into tables correctly that will ensure that no redundancy will occur.
Ex: If there is a combined 'Orders and Drivers' table in the data set each time a new order is entered information such as driver's name and phone number will be stored in the data again and again.
- Prevention of Data Anomalies: it divides into 3 different types:
 1. Update Anomaly: if the driver's name has been entered wrongly, we might need to change hundreds of records.
 2. Insertion Anomaly: a new driver can be added only if they being are assigned to an order.
 3. Deletion Anomaly: If all the orders that a driver delivers have been deleted, the driver data will also be deleted from the system.
- Scalability & Maintainability.

Strong entities: Strong entities are entity types that exist independently.

Order, Customer, Driver and Menu items are Strong entities their existence does not depend on any another entity. Each strong entity has its own unique primary key that fully identifies each record.

Weak entities: any entity that depends on another entity to exist. Most of them do not have a standalone primary key of their own

Payment record, delivery status, order items are weak entities depending on order entity.

Customer address is a weak entity depending on customer entity.

3.3 Logical Design

Definition:

It is a process of converting the conceptual design into a set of structured and relational design that both keys (PK and FK) and attributes are determined, without diving into coding details or hardware system.

Goal of Logical Design:

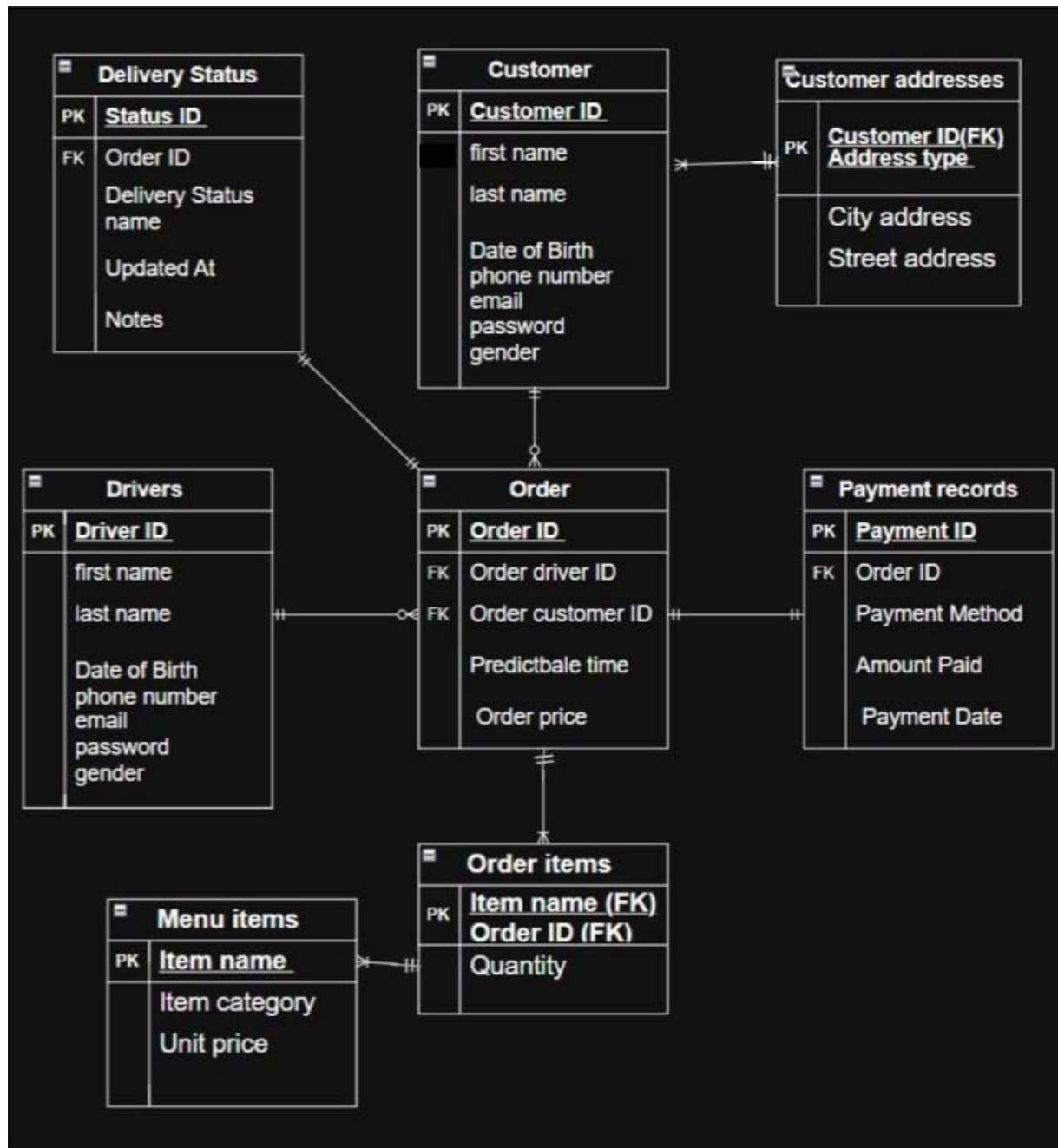
The main goal of the logical design is to go from the general first schema (the conceptional schema) into organized structured properly structured, while ensuring that data will be stored efficiently without mistakes or redundancies.

- Convert each entity into a table and each attribute into a column and present the complex relationships (M: N) in the right way.
- Getting rid of redundancy and mistakes by implementing Normalization forms (the benefits of normalization have already been explained in detail in normalization section)
- Creating a standard global schema without delving into programming details.

Components of Logical Design:

1. Relations (Tables)
2. Attributes: descriptive data fields to each table.
3. Primary Keys (PK): Unique column identifiers that distinguish each record in a relation.
4. Foreign Keys (FK): Attribute columns that reference the primary key of another relation to enforce referential integrity.
5. Integrity Constraints: Relational rules that each table must follow

The logical schema:



Description of the Logical Schema:

1. Customer Table:

It stores customer information it's the primary key is customer ID and it has other attributes such as first name, last name, date of birth, phone number, e-mail password and gender

2. Customer addresses Table:

Derived from complex attribute (customer address). It contains a Composite Primary Key, 'Customer ID' (which is also a Foreign Key referencing to the Customer table) and 'Address type'. It also contains atomic attribute for 'City address' and 'Street address'.

3. Drivers Table:

It stores driver information using 'Driver ID' as the Primary Key (PK). It stores 'first name', 'last name', 'Date of Birth', 'phone number', 'email', 'password', and 'gender'.

4. Order Table:

It Acts as a center of this logical schema, Its primary key is order ID it's also has two foreign keys the first one is customer ID refers to customer, and the second one is driver ID refers to driver table it also contains predictable time and order price attributes

5. Menu items Table:

Stores the catalog of dishes and drinks using 'Item name' as the Primary Key (PK), along with 'Item category' and 'Unit price'.

6. Order items Table:

This table has been created from a multi-valued attribute (Order contents). It uses a Composite Primary Key composed of 'Item name' (FK referencing Menu items) and 'Order ID' (FK referencing Order), alongside the 'Quantity' ordered.

7. Payment records Table:

Implements a 1:1 relationship with orders using 'Payment ID' as the Primary Key (PK) and 'Order ID' as a Foreign Key (FK) referring to the Order table. It contains 'Payment Method', 'Amount Paid', and 'Payment Date'.

8. Delivery Status Table:

This table refers to real-time delivery tracking records it relates to orders with 1: 1 relation also it contains status ID as the primary key and order ID as a foreign key also it contain delivery status name, updates at and notes

3.4 Physical Design

Definition:

It is the process of converting the logical schema into an actual real-world software implementation-ready technical specification, that is designed to a specific system. This process defines data types in the design and constraints used to protect data on physical storage media efficiently.

Goal of Physical Design:

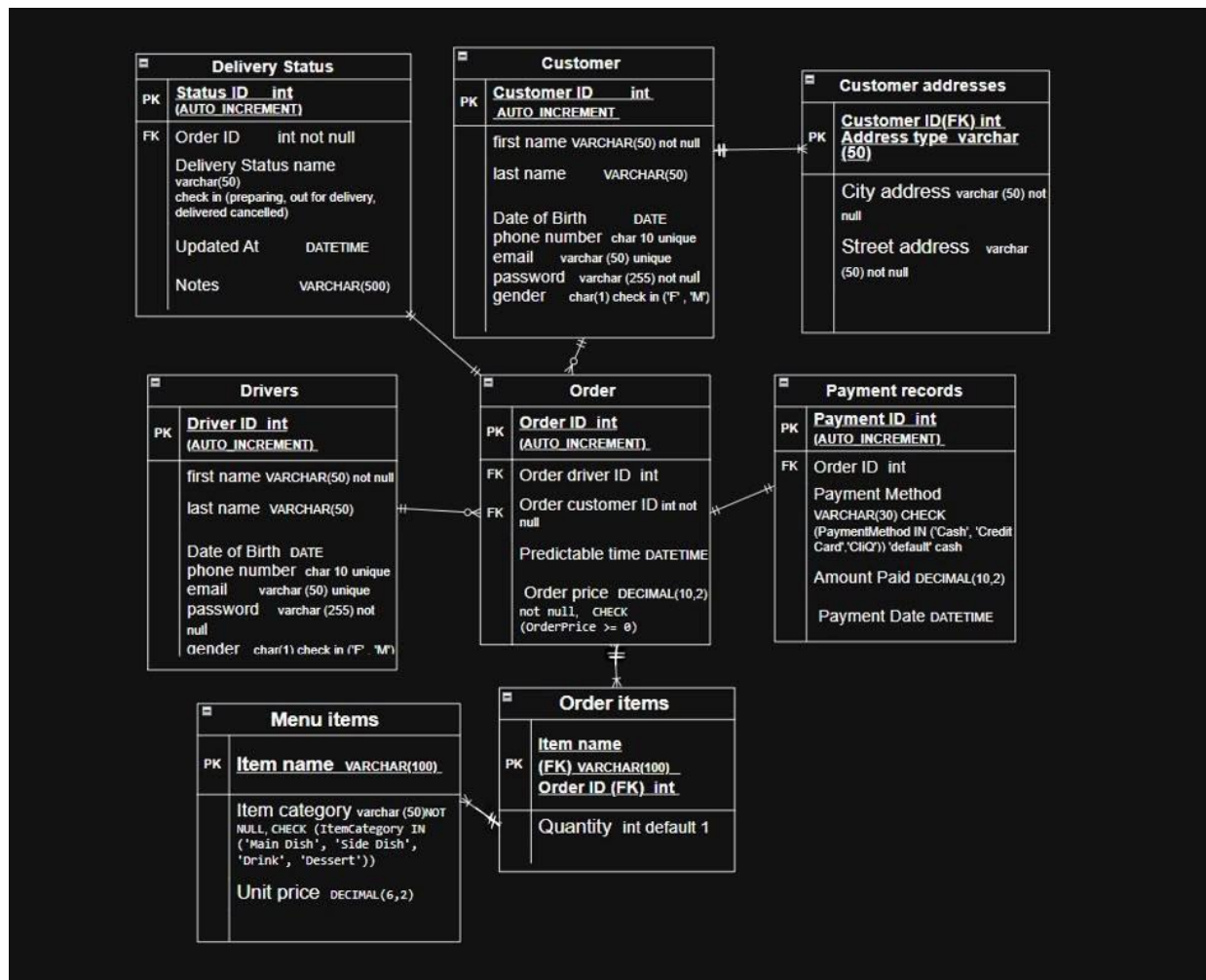
The main goal behind physical design can be summarized into three main points: 1. Performance 2. Storage efficiency 3. Data integrity.

- Precise Data Types & Memory Allocations: choosing the wrong data type causes a big waste of storage and latency in processing.
- Database Constraints: rules that the entered data should follow.
- Improving transaction speed: example on it using `INT AUTO_INCREMENT`

Components of Physical Design:

1. Physical Tables and Columns.
2. Concrete Data Types: Ex: (`VARCHAR(50)`, `INT`, `DECIMAL (10,2)`, `DATETIME`).
3. Domain & Integrity Constraints: Ex: `NOT NULL`, `UNIQUE`.
5. Auto-Incrementing Mechanisms

Physical Design



The physical schema shows the entities and the relationships between them and their attributes with their types and constraints and tables primary key and foreign key all of this after finishing the normalization process.

Explanation of the physical schema:

All the Ids that are primary keys in physical schema are defined as int with AUTO_INCREMENT constraints, this constraints means with each new record being added to tables the Id will be automatically increment by 1.

This ensures that each new record will have a unique Id without forcing the developer to track the Id values. The INT data type stands for integer and it is a numeric value.

Decimal is also a numeric value, and the first value in its parentheses refers to the number of digits the whole number contains, while the second value represents the number of digits after

the decimal point. Example, the order price decimal (10, 2) means the price contains up to 10 digits and two of those ten locate after the decimal point..

Data Types Used in the Schema:

VARCHAR (Variable Character):

This data type holds a variable-length string with a pre-defined maximum length in parentheses. This data type varies from fixed-length data types, since it dynamically allocates memory only for what is actually entered. For example, VARCHAR(50) has been allocated to fields like First Name, Last Name and City Address, which require variable names with a limited number of characters. On the other hand, VARCHAR(255) has been allocated to the Password field to securely hold cryptographic hash passwords. Lastly, VARCHAR(500) has been allocated to optional long descriptions such as the Notes field of the Delivery Status table.

CHAR (Fixed Character):

This data type holds fixed-length strings. This data type allocates exactly the same space as defined by the database for each record, regardless of how many characters are actually entered. In the current schema, CHAR(1) has been allocated to the Gender field, since it can be just one character ('M' or 'F'). In addition, CHAR(10) has been allocated to Phone Number fields.

DATE vs. DATETIME:

DATE data type saves calendar dates in YYYY-MM-DD format without time. It is used in cases when there is no need for time tracking for certain attributes such as Date of Birth in the Customer and Drivers tables. Meanwhile, the DATETIME data type includes both the calendar date and the exact time in format YYYY-MM-DD HH:MM:SS. The latter is used for the attributes where time tracking is important and includes Predictable Time attribute in the Order table, Payment Date in the Payment Records table and Updated at in the Delivery Status table.

Database Constraints and Key Design:

Primary Key (PK) and Composite Primary Key:

A Primary Key is unique identifier of each individual record in the table and does not allow NULL value. Single-entity tables such as Customer, Drivers, Order have one PK - single ID field. The junction and entity-relationship tables, namely Order Items and Customer Addresses,

have Composite Primary Key in form of two fields (Item Name + Order ID) and (Customer ID + Address Type) respectively.

Foreign Key (FK):

The Foreign Key creates a referential link between two tables. For instance, Order Customer ID from the Orders table is used as a Foreign Key that refers to Customer ID in the Customers table. The key maintains the referential integrity by avoiding an order for a non-existent customer and avoids the possibility of orphaned entries.

NOT NULL Constraint:

It ensures completeness by preventing the null values, this means that these columns must be filled in at the point of creation. The constraint is applied to the First Name, Password and City Address attributes which are crucial to the operations.

UNIQUE Constraint:

It is a constraint that ensures that every value in this column is a unique value and this helps in avoiding duplication. In the Customers and Drivers tables, the constraint is applied to the Email and Phone Number attributes, because we cannot have two people have the same email or phone number.

CHECK Constraint:

That's mean the entered value has to follow a specific rule in order to be considered valid. This ensures the accuracy of the entered data.

Examples:

- Gender Validation: CHECK (gender IN ('F', 'M'))
- Menu Classification: CHECK (ItemCategory IN ('Main Dish', 'Side Dish', 'Drink', 'Dessert'))
- Payment Methods: CHECK (PaymentMethod IN ('Cash', 'Credit Card', 'CliQ'))
- Financial Values: CHECK (OrderPrice >= 0)

Default constraint:

It is responsible to assign a value to a column if no value has been provided in the insert operation. An example of a DEFAULT constraint is Quantity in the Orders Items table that is assigned the value of 1 through DEFAULT 1.

3.5 Effectiveness of the design

Jasmine Restaurant delivery management system has been developed to organize delivery management by following organized approach to design the data and to cover all the operational processes for the delivery system. The data sets in its final version contain eight tables. Each table has its purpose in existing in the system. Order table, for example, is center of the data schema. It connects most tables together and it has important information about the order. There

is also order items table, which is the table containing the items in each order, these two tables are connected with each other by a foreign key. There is also menu items table, which stores all the items in the restaurant menu alongside its category and unit price. Drivers table contains the driver's information about who is responsible for delivering the order, while customer table contains customer information and customers' contact details.

Customer address table is a table that contains customer addresses, the type of customer address and its details, which helps to know the customer location when delivering the orders. Payment records are very important for tracking the restaurant income and managing the financial section of the restaurant.

Specific attributes were chosen for functional roles, such as Phone and Email for communication and uniqueness, Password for secure authentication, PredictableTime for real-time tracking, and UnitPrice to calculate order totals.

In the dataset, tables connect with each other in different relationship types. Order connects with the drivers table with one-to-many relationship because each order can be delivered by one and only one driver, while each driver can deliver multi-orders or zero order if he was absent for example on that day. The relationship between order table and customer table is also one-to-many relationship. Each customer can order many orders or zero orders, while each order should be ordered by one customer and only one customer.

Order table relates to delivery status with one-to-one relationship. Each order can have only one delivery status, and every delivery status record can only belong to one order. Also, order table relates to payment records table with one-to-one relationship. Each order can have one payment record, and each payment record should belong only to one order. It is impossible to have a payment record for two different orders. Order table relates to order items with one-to-many relationship. Each order can contain multi-items or at least one single item, while each order item can be in one order.

However, The relationship from Menu_Items to Order_Items is One-to-Many, where one menu item can appear across multiple order item records, while each order item record refers to exactly one menu item

The relationship between order_items table and menu items is one to many. Each order items can include multi menu items or one item, and each order item refers to one menu item, a single order item can refer to only one menu item.

We cannot implement many-to-many relationship between two tables without creating a table that connects these tables together like we did when we create order items table which connect order table and menu items.

Conceptual design is a very useful and simple design that's easy to understand and read. It gives the developers a general overview about the dataset and it explain the relationships between the entities without going into coding details or physical materials.

Creating conceptual design helps me to understand the main concepts of Jasmine Restaurant Delivery Management System database and helps me in starting this database and making the logical design after it much easier step. The target audience for conceptual design includes the

end users who will interact with the final product, internal stakeholders like project managers and engineers along with the owners of the restaurant.

Transitioning to the logical design transformed conceptual abstractions into concrete relational schema models. In this phase, formal relational identifiers were introduced by defining Primary Keys (PK) and Foreign Keys (FK) across all tables to enforce referential integrity. Complex and multi-valued attributes identified during conceptualization were converted into standalone relations. Specifically, the complex customer address attribute was decomposed into the Customer_Addresses table with a composite primary key (CustomerID, AddressType), allowing a customer to store multiple delivery destinations. Furthermore, the junction table Order_Items was logically defined with a composite primary key (OrderID, ItemName), creating valid relational links between orders and the menu catalog.

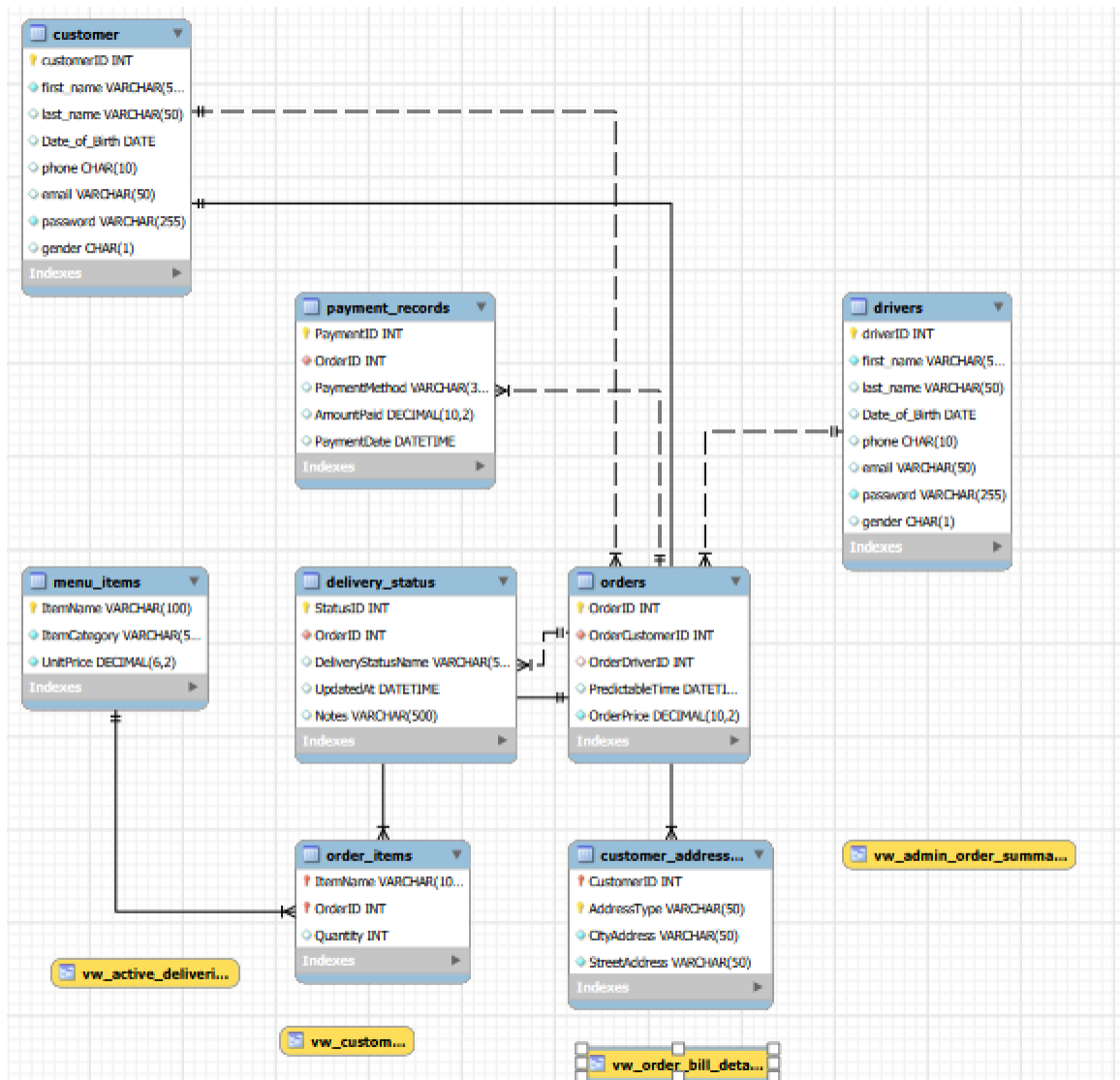
The physical design adapted the logical model to physical storage in the MySQL database engine, prioritizing computational performance, storage efficiency, and data integrity. Concrete data types were allocated based on operational demands. VARCHAR(50) was assigned to dynamic text fields like First Name and City Address to conserve physical disk space, while VARCHAR(255) was dedicated to Passwords to accommodate cryptographic hashes securely. Fixed CHAR(1) was designated for Gender to consume exactly one byte, and CHAR(10) was applied to Jordanian mobile numbers to maintain uniform storage. DECIMAL(10,2) was assigned to OrderPrice and AmountPaid to prevent the floating-point inaccuracies typical of binary numbers in financial ledgers, and DATETIME was allocated to timestamps like PredictableTime and UpdatedAt for scheduling accuracy. Essential database constraints were integrated: AUTO_INCREMENT on integer primary keys to accelerate B-tree indexing; NOT NULL on mandatory fields to prevent incomplete transactions; UNIQUE constraints on phone numbers and emails to avoid duplicate registrations; and CHECK constraints to enforce domain validity directly at the storage level, restricting Gender to ('F', 'M'), ItemCategory to defined categories, and requiring OrderPrice to be non-negative.

A rigorous normalization process was executed up to the Third Normal Form (3NF) to eliminate data redundancy and prevent data anomalies. In the First Normal Form (1NF), atomicity was enforced by splitting composite names into first_name and last_name, deleting the derived Age attribute in favor of Date_of_Birth, and isolating multi-valued order items and complex addresses into separate relations. Applying the Second Normal Form (2NF) eliminated partial dependencies in composite key relations: within Order_Items, attributes such as ItemCategory and UnitPrice depended only on ItemName rather than the complete composite key (OrderID, ItemName), which prompted the extraction of the independent Menu_Items table. Similarly, customer phone details were removed from Customer_Addresses to ensure that non-key attributes depend on the entire composite key. In the Third Normal Form (3NF), transitive dependencies were systematically removed; courier and customer contact details were decoupled from the central Orders table, ensuring that attributes depend solely on their respective primary keys (DriverID or CustomerID) rather than transitively on OrderID, thereby eliminating insertion, update, and deletion anomalies.

The culmination of these systematic design stages directly achieves the ultimate goal of fulfilling both user and system requirements. From a user perspective, administrators obtain end-to-end dispatch visibility through dedicated views, couriers receive clean delivery rosters without exposure to sensitive customer credentials, and customers experience reliable ordering and order

tracking. From a system perspective, the relational structure guarantees data integrity, rapid sub-second query execution, consistent backup routines, and scalable multi-user operations. This effectively modernizes Jasmine Restaurant's operations by replacing disorganized paper records and manual spreadsheets with an enterprise-grade relational database system.

5 Physical Schema



6 Database Development

6.1 Database Overview

Tables:

Tabel	Name	Disruption
1	Customer	Purpose: store customer information, contact details. Stored Attributes & Data Types: customerID (INT, AUTO_INCREMENT), first_name (VARCHAR(50), NOT NULL), last_name (VARCHAR(50)), Date_of_Birth (DATE), phone (CHAR(10), UNIQUE), email (VARCHAR(50), UNIQUE), password (VARCHAR(255), NOT NULL), gender (CHAR(1)). Constraints & Rationale: first_name and password use NOT NULL to ensure account accessibility and authentication completeness. phone and email are enforced with UNIQUE constraints because there are not two people who can have the same email or phone number. gender is restricted via CHECK (gender IN ('F', 'M')) to preserve domain accuracy. Primary Key (PK): customerID. Foreign Keys (FKs): None (Independent Strong Entity).
2	Customer_Addresses	Purpose: Resolve the multi-valued and composite customer address attribute (complex value), allowing the customer to store multi locations details. Stored Attributes & Data Types: CustomerID (INT), AddressType (VARCHAR(50)), CityAddress (VARCHAR(50), NOT NULL), StreetAddress (VARCHAR(50), NOT NULL). Constraints & Rationale: CityAddress and StreetAddress use NOT NULL because the driver needs to know the actual customer location to deliver the orders. Primary Key (PK): Composite Primary Key (AddressType, CustomerID), which prevents duplicate address labels for the same customer Foreign Keys (FKs): CustomerID references Customer(customerID). This ensure the address integrity by ensuring that this address related to a real registered customer.

3	Drivers	Purpose: stores employees records, information and contact details. Stored Attributes & Data Types: driverID (INT, AUTO_INCREMENT), first_name (VARCHAR(50), NOT NULL), last_name (VARCHAR(50)), Date_of_Birth (DATE), phone (CHAR(10), UNIQUE), email (VARCHAR(50), UNIQUE), password (VARCHAR(255), NOT NULL), gender (CHAR(1)). Constraints & Rationale: NOT NULL enforces necessary identification on first_name and password which always him to access his account. UNIQUE ensures phone numbers and employee emails are unique. gender can be only female and male by CHECK (gender IN ('F', 'M')). Primary Key (PK): driverID uniquely identifies each driver in the system. Foreign Keys (FKs): None (Independent Strong Entity).
4	Menu_Items	Purpose: stores menu catalog for all available dishes that restaurants can serve. Stored Attributes & Data Types: ItemName (VARCHAR(100)), ItemCategory (VARCHAR(50), NOT NULL), UnitPrice (DECIMAL(6, 2), NOT NULL). Constraints & Rationale: ItemCategory CHECK (ItemCategory IN ('Main Dish', 'Side Dish', 'Drink', 'Dessert')) to ensure that each dish is within these 4 categories. UnitPrice is secured with CHECK (UnitPrice >= 0) because price can not be negative number. Primary Key (PK): ItemName serves as the primary key since dish names are distinct across the menu. Foreign Keys (FKs): None (Independent Strong Entity).
5	Orders	Purpose: it is the central entity, connecting customers and drivers. Stored Attributes & Data Types: OrderID (INT, AUTO_INCREMENT), OrderCustomerID (INT, NOT NULL), OrderDriverID (INT), PredictableTime (DATETIME), OrderPrice (DECIMAL(10, 2), NOT NULL).

		Constraints & Rationale: OrderCustomerID is set to NOT NULL because an order cannot exist without a customer. OrderDriverID allows NULL values to represent newly placed orders that yet is not assigned to any driver. OrderPrice is restricted by CHECK (OrderPrice >= 0) it cannot be negative Primary Key (PK): OrderID uniquely identifies each order Foreign Keys (FKs): 1. OrderCustomerID references Customer(customerID) to enforce customer existence. 2. OrderDriverID references Drivers(driverID) to assign orders directly to drivers.
6	Order_Items	Purpose: solve many to many relationships (N: M) between orders and Menu items. And, sorting the orders items dishes alongside with their quantities. Stored Attributes & Data Types: ItemName (VARCHAR(100)), OrderID (INT), Quantity (INT, DEFAULT 1). Constraints & Rationale: Quantity uses DEFAULT 1 to ensure at least one unit is counted per order. Primary Key (PK): Composite Primary Key (ItemName, OrderID) to prevent duplicate item entries inside the same order. Foreign Keys (FKs): 1. OrderID references Orders(OrderID), the order which contain these items 2. ItemName references Menu_Items(ItemName), any order item should be existed in the restaurant menu.
7	Payment_Records	Purpose: keep tracking and storing the financial orders prices alongside with payment methods and amounts. Stored Attributes & Data Types: PaymentID (INT, AUTO_INCREMENT), OrderID (INT, NOT NULL, UNIQUE), PaymentMethod (VARCHAR(30), DEFAULT 'Cash'), AmountPaid (DECIMAL(10, 2)), PaymentDate (DATETIME).

		Constraints & Rationale: OrderID is defined as UNIQUE to strictly enforce the One-to-One (1:1) relationship between Orders. Payment Method should be one following CHECK (PaymentMethod IN ('Cash', 'Credit Card', 'CliQ')) the restaurant don't deal with any payment records out these 3 And, DEFAULT 'Cash'. Amount Paid is verified by CHECK (AmountPaid >= 0). Primary Key (PK): PaymentID Foreign Keys (FKs): OrderID references Orders(OrderID), because payment record can not exist without an order.
8	Delivery_Status	Purpose: Tracks a real time order, showing its state and the arriving predictable time. Stored Attributes & Data Types: StatusID (INT, AUTO_INCREMENT), OrderID (INT, NOT NULL, UNIQUE), DeliveryStatusName (VARCHAR(50)), UpdatedAt (DATETIME), Notes (VARCHAR(500)). Constraints & Rationale: OrderID is enforced with UNIQUE to maintain a 1:1 link per order. DeliveryStatusName is validated via CHECK (DeliveryStatusName IN ('preparing', 'out for delivery', 'delivered', 'cancelled')). Primary Key (PK): StatusID Foreign Keys (FKs): OrderID references Orders(OrderID), to know the order that need to be track.

Views:

Views	Name	Description
1	vw_Admin_Order_Summary	Aim: Giving managers a quick summary of each order, it's the customer who orders it and the driver who will deliver it. Implementation: Using join tool to connect between customer table, orders table and drivers table. Using the foreign keys in the order table (OrderCustomerID, OrderDriverID) to be connected with customer and drivers tables primary keys.

		Note** while writing a code two columns has the same name and they are (first_name) for customer and (first_name) for driver this confused the program and raise an error preventing the code from working. To fix this problem I have given each column a nick name using as so the new names for the two-column become (customer first name, driver first name) Target User: Restaurant Administrators and Operations Dispatchers
2	vw_Active_Deliveries	Aim: allowing drivers to see a quick summery containing information about the deliveries assigned to them and customer name and contact details (phone number), order id, the order price, payment method and the amount paid. All this information makes it much easier for drivers to find most information needed at once. Implementation: Connecting 5 tables together using join tool these tables are: Orders, Customer, Drivers, Payment_Records, and Delivery_Status. This connection happens by using primary keys and foreign keys as order foreign keys that are used to connect orders with drivers and customers (OrderCustomerID, OrderDriverID), and Payment_Records foreign key which used to connect it with orders table (OrderID). And DeliveryStatusName foreign key which connect it with orders table as well. The only records will be shown are records with 'preparing' or 'out for delivery' delivery status, because driver don't need to see cancelled or delivered orders. Target User: Delivery Drivers and Dispatch Supervisors.
3	vw_Order_Bill_Details	Aim: building a detailed bill for the customer containing the order number, the items, its category and price.

		Implementation: connecting using join, Orders table, order_item table, Menu_items table. By using the foreign keys in order_item table (OrderID, ItemName) Target User: Customers, Kitchen Preparation Staff
4	vw_customer	Aim: help customers tracking their orders in real time and contacting the drivers Implementation: Integrates Customer, Orders, Order_Items, and Delivery_Status on their respective identifiers, paired with a LEFT JOIN on Drivers to ensure that customers can still view tracking information even when an order is still in preparation and a driver has not yet been assigned. Target User: Registered Customers.

Procedure

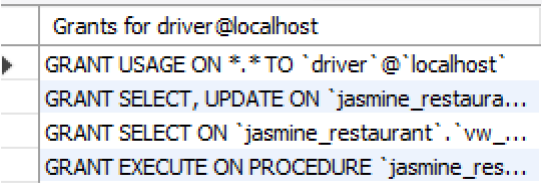
Procedure	Name	Description
1	p_AssignDriverToOrder	Aim: it is procedure that helps in assigning orders to drivers and it also changes order status to be 'out for drive'. Implementation: The procedure takes two input attributes (orderID int, driverID) and then connects the driver record in drivers table with a certain order in orders table, this happens by making the foreign key that connects these two tables have the wanted drive key. Also, the procedure changes the order status to be 'out for drive' from drive status table. It also update the 'updatedat' attribute to be now() → the time of updated Target User: drivers.
2	p_new_customer_order	Aim: helping customers in creating new orders in fast and easy way. This procedure fills the record in orders table and creates records in payment_record table and in Delivery_Status, order_items and all these records related to new order.

		Implementation: This procedure has 6 input parameters, it starts with creating a record in orders table and then saving that order id in a local variable (p_newOrderID) by using LAST_INSERT_ID which return the last insert ID in the data set. Then it uses the order id which has been saved in variable alongside with input parameters to create records in payment_records, Delivery_Status, order_items tables. The DeliveryStatusName will be 'preparing' for all the new orders and the updated time will be assigned using Now() function. Target User: Registered Customers
3	p_UpdateMenuItemPrice	Aim: it allows the admin updating the price for a certain item in the restaurant menu. Implementation: Two parameters have been used in this procedure. And update command has been used to change the price, while where Clause created a condition to ensure not all records in menu_items will be changed. Target User: Restaurant Management and Pricing Controllers.
4	p_GetCustomerOrderCount	Aim: counting the number of orders a customer has ordered. Implementation: This procedure takes one input parameter and returns one output parameter. It uses count function to count the number of orders that contain a chosen customer id and use (into) clause to save the value in the output parameter Target User: Managers and Administrators.

6.2 Security

Username	Privilege Command	Description	Screenshot
----------	-------------------	-------------	------------

admin	GRANT ALL PRIVILEGES ON jasmine_restaurant.* TO 'admin'@'localhost';	Grant used to give a certain privilege to user. The admin should have full control over and access to the data in the system. By using grant with all privileges and the (*), we had given the user full control to see, update, insert and delete the data. It also gives the admin the ability to create, drop and modify the dataset tables. Which is ability the admin will not have if this command (GRANT SELECT, INSERT, UPDATE, DELETE ON jasmine_restaurant.* TO 'admin'@'localhost';) Written instead. In addition, the admin has full access to the views and procedures.	<pre>-- identifying admin user select user (); drop user if exists 'admin'@'localhost'; create user if not exists 'admin'@'localhost' identified by '123@admin'; -- give all the privileges to the admin GRANT ALL PRIVILEGES ON jasmine_restaurant.* TO 'admin'@'localhost'; show grants for 'admin'@'localhost';</pre> <table><tr><td></td><td>Grants for admin@localhost</td></tr><tr><td>▶</td><td>GRANT USAGE ON *.* TO 'admin'@'localhost'</td></tr><tr><td></td><td>GRANT ALL PRIVILEGES ON `jasmine_restaurant`...</td></tr></table>		Grants for admin@localhost	▶	GRANT USAGE ON *.* TO 'admin'@'localhost'		GRANT ALL PRIVILEGES ON `jasmine_restaurant`...
	Grants for admin@localhost								
▶	GRANT USAGE ON *.* TO 'admin'@'localhost'								
	GRANT ALL PRIVILEGES ON `jasmine_restaurant`...								
Driver	1)grant select on jasmine_restaurant.vw_active_deliveries to 'driver'@'localhost';	1)It gives the driver a read-only accesses a vw_active_deliveries View. This view allows drivers to see a quick summary containing information about the deliveries assigned to them. It is very useful permission to be given to the drivers which will help them in their job, without putting the company sensitive data in danger.	<pre>-- driver Privileges grant select on jasmine_restaurant.vw_active_deliveries to 'driver'@'localhost'; grant execute on procedure jasmine_restaurant.p_AssignDriverToOrder to 'driver'@'localhost'; grant select, update on jasmine_restaurant.delivery_status to 'driver'@'localhost'; show grants for 'driver'@'localhost';</pre>						

	2)grant execute on procedure jasmine_restaurant.p_AssignDriverToOrder to 'driver'@'localhost'; 3) grant select, update on jasmine_restaurant.delivery_status to 'driver'@'localhost';	2)Authorizes the courier to invoke the stored procedure that claims unassigned orders and synchronizes transit delivery statuses. 3)Having the ability to see and modify delivery_status table, which allows drivers to update the order status and record delivery notes.	 <pre> Grants for driver@localhost GRANT USAGE ON *.* TO 'driver'@'localhost' GRANT SELECT, UPDATE ON `jasmine_restaura... GRANT SELECT ON `jasmine_restaurant`.`vw_... GRANT EXECUTE ON PROCEDURE `jasmine_res... </pre>
customer	1)grant select on jasmine_restaurant.menu_items to 'customer'@'localhost'; 2) grant select on jasmine_restaurant.vw_customer to 'customer'@'localhost';	1) it gives the customer read-only access to view the restaurant menu. This will help the customer search for the dishes he wants to order and their prices and categories. 2)Grants access to track order progress, estimated arrival times, and assigned courier details securely through an integrated view.	<pre> grant select on jasmine_restaurant.menu_items to 'customer'@'localhost'; grant select on jasmine_restaurant.vw_customer to 'customer'@'localhost'; grant select on jasmine_restaurant.vw_Order_Bill_Details to 'customer'@'localhost'; grant execute on procedure jasmine_restaurant.p_new_customer_order to 'customer'@'localhost'; show grants for 'customer'@'localhost'; </pre>

	3) GRANT SELECT ON jasmine_restaurant. vw_Order_Bill_ Details TO 'customer'@ 'localhost'; 4) GRANT EXECUTE ON PROCEDURE jasmine_restaurant.p_new_customer_order TO 'customer'@'localhost';	3) Provides read access to the itemized billing view, enabling customers to verify dish breakdowns and unit costs for their orders. 4) Gives execution rights to place new food orders, triggering automated billing, tracking, and item association through the procedural workflow.	Grants for customer@localhost <pre>GRANT USAGE ON *,* TO 'customer'@'localh... GRANT SELECT ON `jasmine_restaurant`.`men... GRANT SELECT ON `jasmine_restaurant`.`vw_... GRANT SELECT ON `jasmine_restaurant`.`vw_... GRANT EXECUTE ON PROCEDURE `jasmine_res...</pre>
--	--	---	---

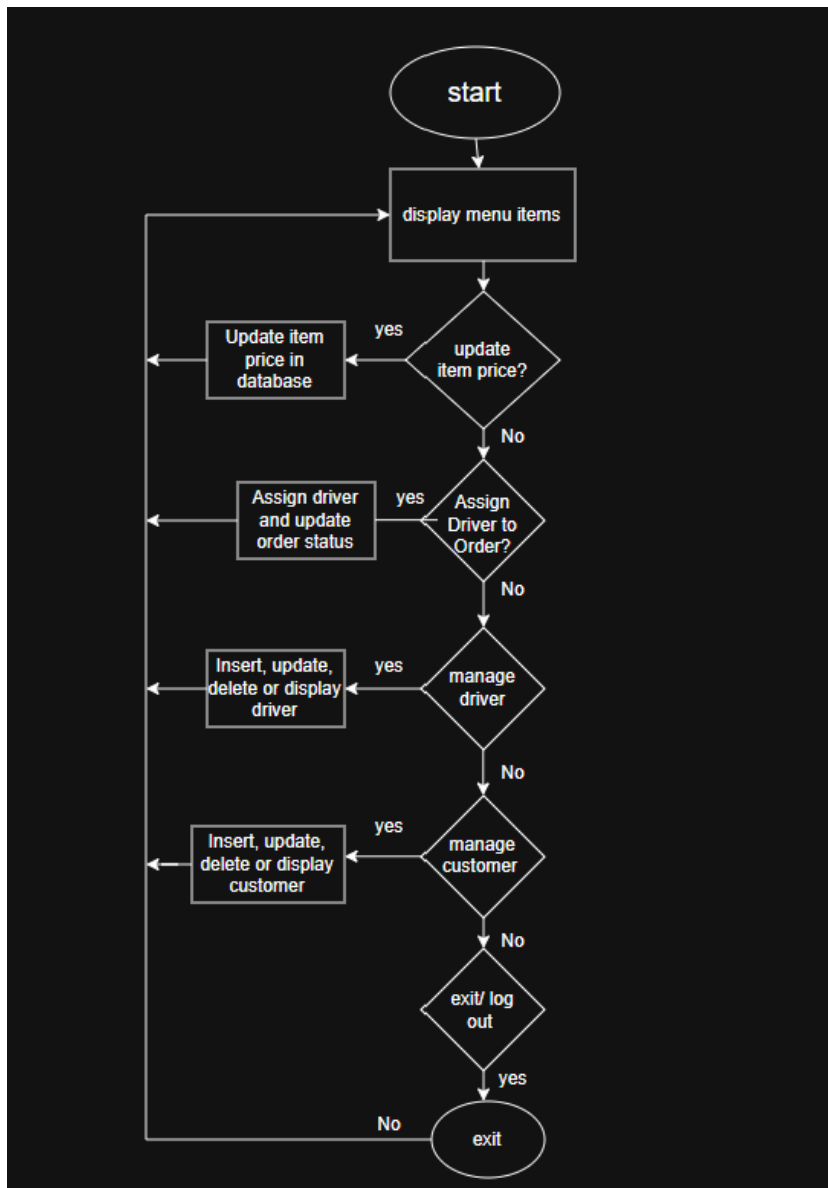
6.3 User Interface

6.3.1 Flowchart and Data Movement Diagrams

Flowchart: A set of standard symbols connect in a certain way to represent a process or workflow showing the sequence of steps descriptions and operations.

In order to understand the workflow for the admin in Jasmine restaurant delivery management and make it easy to read we have made a flowchart to represent this process visually.

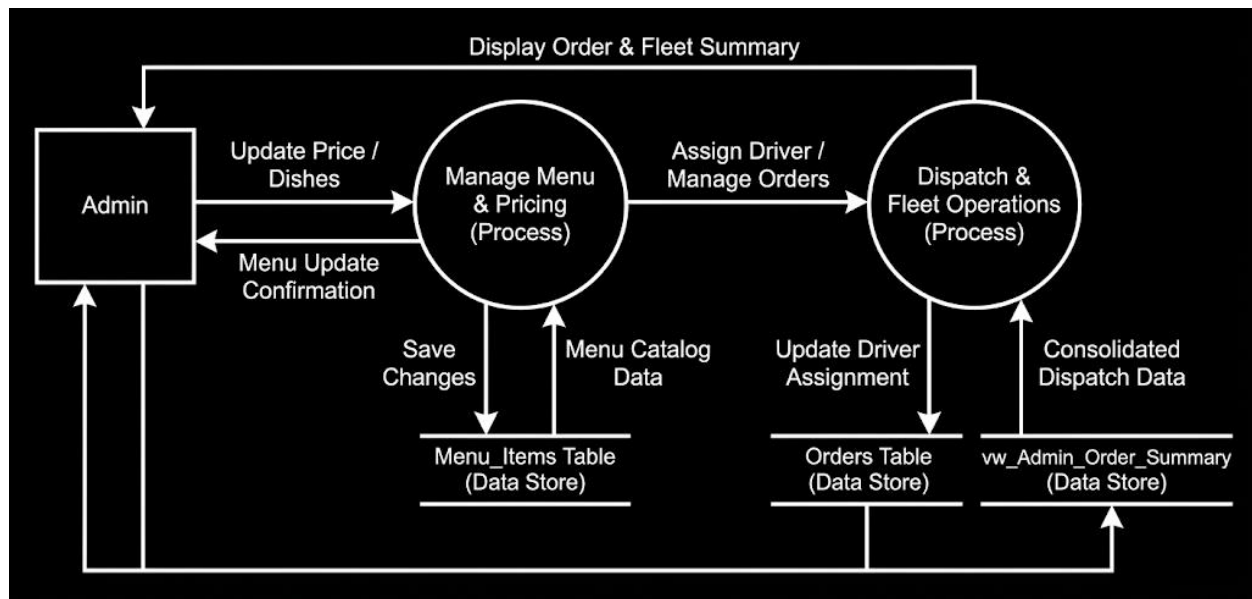
Admin flowchart:



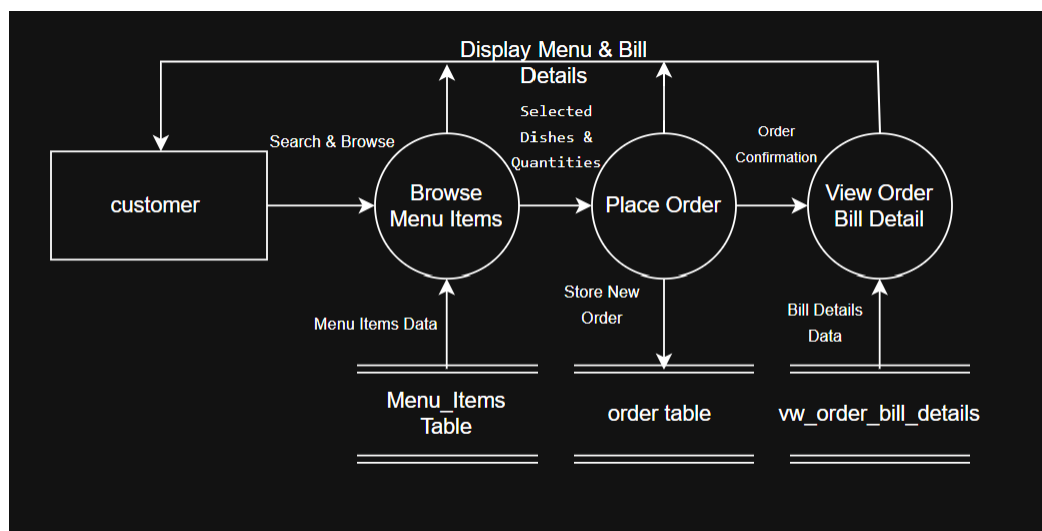
Data Movement Diagram:

while flow chart focuses on the sequence of steps, decisions and operations on the other hand data movement diagram focuses on how the data moves and flows within the system

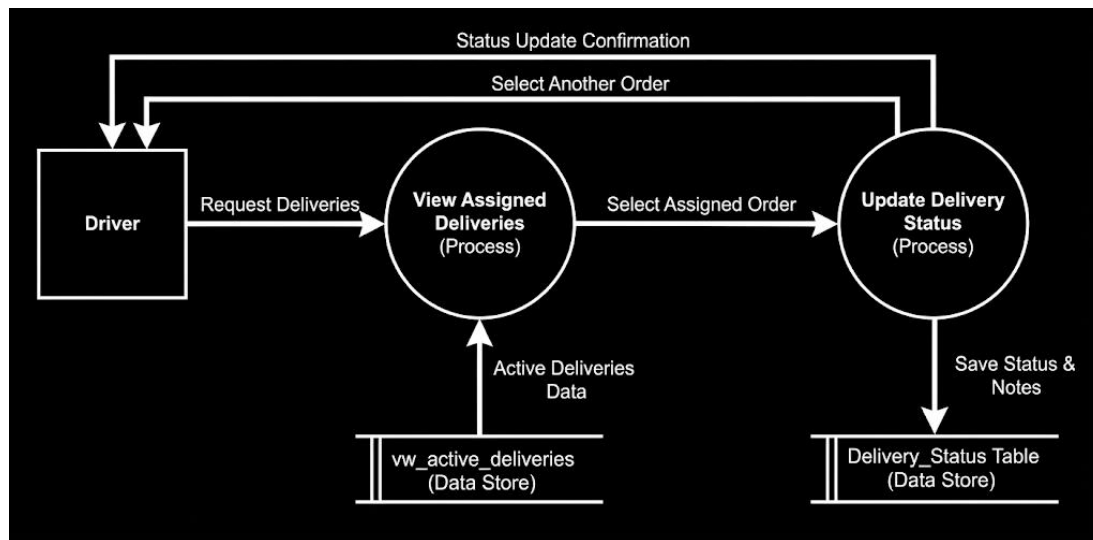
DFD for the admin:



DFD for the customer:



DFD for the driver:



6.3.2 Interfaces Development

Page ID	Title	Description	Screenshot
1	Customer Management Interface	It provides a console to manage customer profiles, including contact numbers, birth date. Create: register new customer account into the database by (Add new) button View the current customers details by clicking on view icon on each customer record Update: by clicking on the pencil icon to modify customer records Delete: Remove inactive accounts using the delete icon (X).	

2	Delivery Drivers Fleet Management	Having full control of driver records. Like the privileges in the customer page the admin using the driver page can add new driver, update certain driver information, delete the records of any driver who no longer works at the restaurant if needed. And of course, being able to access all drivers personal details. Reporting: Export driver rosters for shift scheduling and administrative reporting.	System Drivers Menu Items Orders View Orders Order Summary Drivers
---	-----------------------------------	---	---

5	Consolidated Order Summary	An integrated multi-table reporting console powered by the database view vw_admin_order_summary, joining Customer, Orders, and Drivers to provide real-time dispatch visibility. Here the admin can read and search and filter. But the admin can not insert and delete in this page.	Vw Admin Order Summary 1 Refresh Export Print Quick search <table><thead><tr><th>☐</th><th>Order ID</th><th>Customer ID</th><th>Customer First Name</th><th>Driver ID</th><th>Driver First Name</th></tr></thead><tbody><tr><td>☐</td><td>1</td><td>1</td><td>Ahmad</td><td>1</td><td>Tariq</td></tr><tr><td>☐</td><td>2</td><td>1</td><td>Ahmad</td><td>2</td><td>Khaled</td></tr><tr><td>☐</td><td>3</td><td>2</td><td>Sarah</td><td>2</td><td>Khaled</td></tr><tr><td>☐</td><td>4</td><td>3</td><td>Omar</td><td>2</td><td>Khaled</td></tr></tbody></table>	☐	Order ID	Customer ID	Customer First Name	Driver ID	Driver First Name	☐	1	1	Ahmad	1	Tariq	☐	2	1	Ahmad	2	Khaled	☐	3	2	Sarah	2	Khaled	☐	4	3	Omar	2	Khaled
☐	Order ID	Customer ID	Customer First Name	Driver ID	Driver First Name																												
☐	1	1	Ahmad	1	Tariq																												
☐	2	1	Ahmad	2	Khaled																												
☐	3	2	Sarah	2	Khaled																												
☐	4	3	Omar	2	Khaled																												

7 Maintenance

7.1 Database recovery & backups

In relational database systems, having a backup is not an optional step, but it is very essential way to protect the database from any unpredictable occasions that might ruin the data or make the system unable to do its job. In Jasmine Restaurant the delivery service is an active service the restaurant provides and one of the main sites income the restaurant depends on comes from delivered orders. Therefore, any prolonged work stoppage would result in a significant loss for the restaurant.

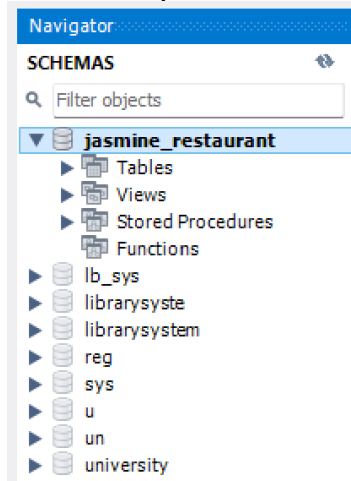
According to Elmasri and Navathe (2016, p. 753), a Database Backup is “the process of periodically creating duplicate copies of the entire database and its transactional log files, storing them securely on independent offline storage media.” Within the operational workflow of Jasmine Restaurant, systematic backups safeguard business assets against accidental record deletions, drive crashes, and malicious software attacks.

In contrast, Database Recovery involves the programmatic reconstruction of corrupted or lost data. As defined by Silberschatz, Korth, and Sudarshan (2020, p. 817), recovery represents “the set of techniques and protocols dedicated to restoring the database to a consistent state that existed just prior to a failure, preserving transaction atomicity and durability.” Implementing automated recovery minimizes operational downtime (Recovery Time Objective - RTO) and prevents lost transactions (Recovery Point Objective - RPO), allowing the restaurant to maintain active delivery dispatching without disruption.

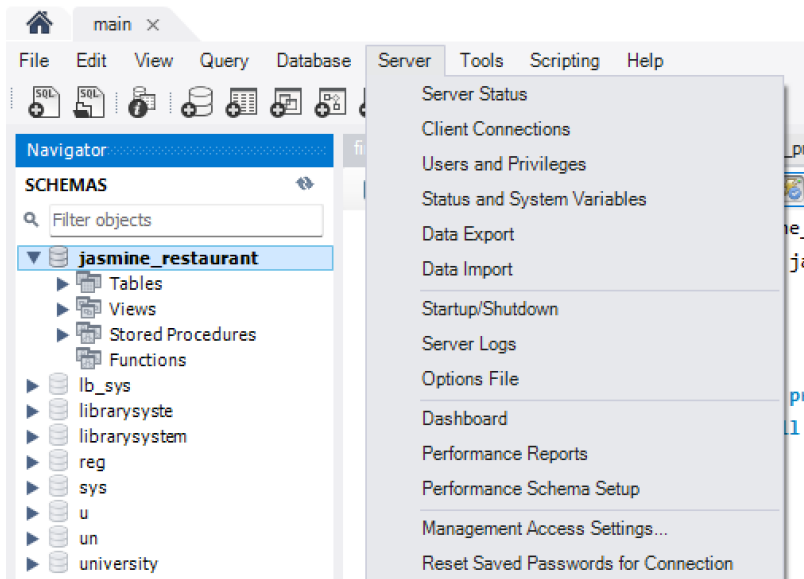
Export Process

To establish a backup of the Jasmine Restaurant delivery management system, logical database dump was generated by following these steps below:

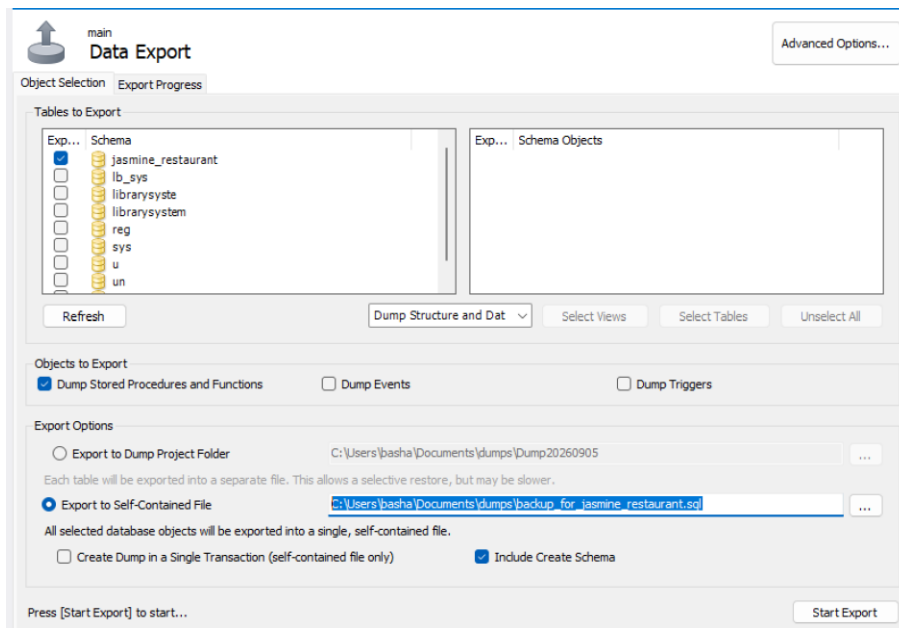
- 1) The first step is to choose the jasmine_restaurant database from here:



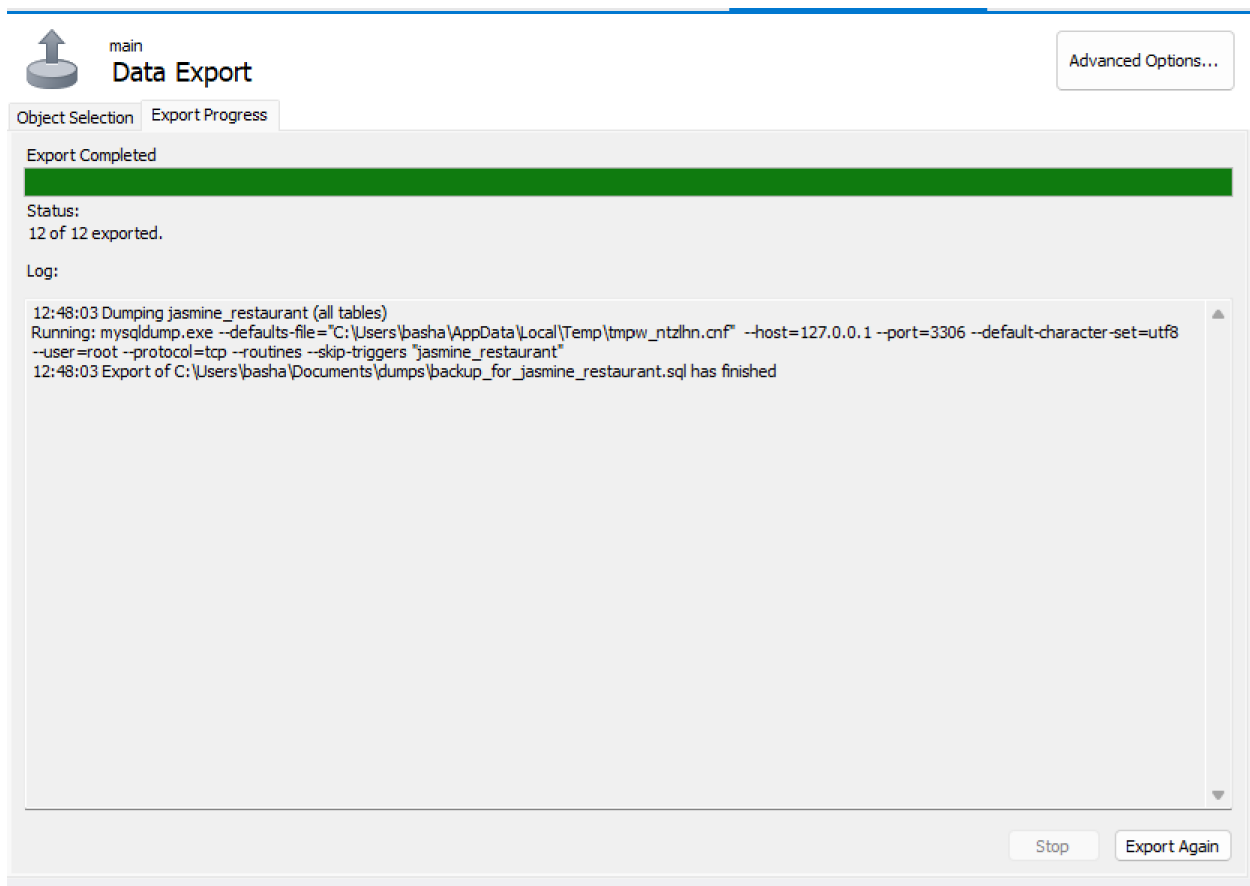
- 2) Go to server tab from the page top bar then click on 'Data Export' from the drop-down menu:



- 3) A data export page will appear from it we choose Dump Stored procedure and function to include stored procedures in the backup.
- 4) Click on 'Export to Self-Contained File' if we want to encapsulate every thing (tables, views and procedures) in one file
- 5) Chose 'Include Create Schema' to make the backup file Fully Portable, and automatically create a schema
- 6) And chose to export Dump Structure and Data to export the columns its keys, constraint and data.



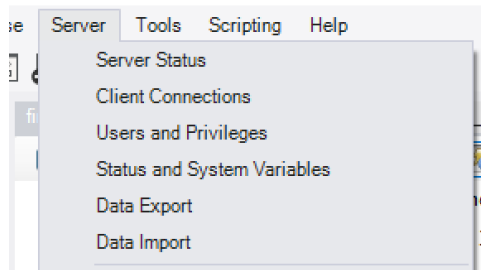
7) Click on 'Start Export' and this page will appear



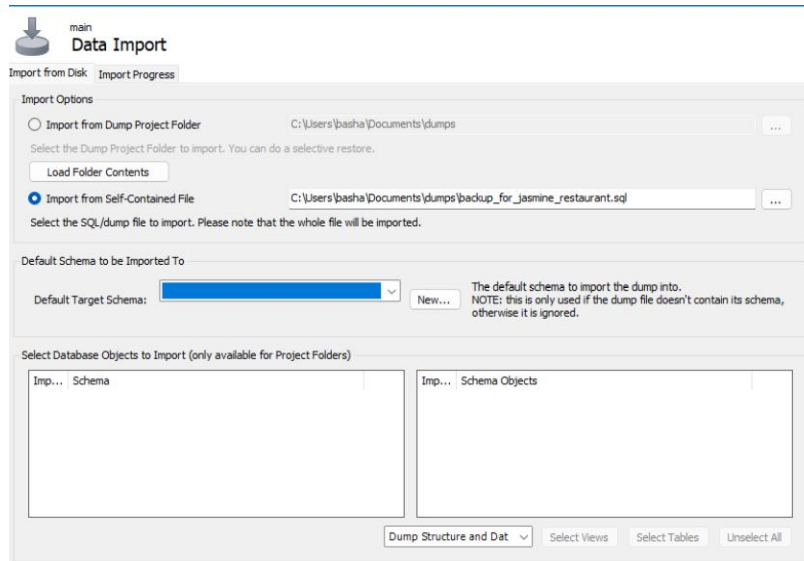
Import Process

Now there is a backup saved for the Jasmine Restaurant delivery management system that we can import when any problem happens to the data, by the following steps:

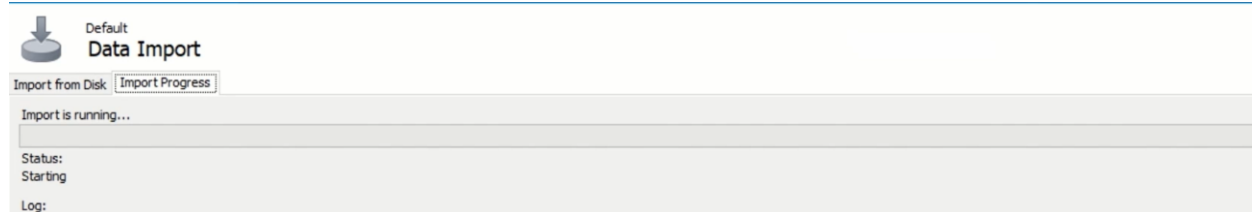
- 1) Go to server tab from the page top bar then click on 'Data import from the drop-down menu:



- 2) Select the database path



- 3) Click 'Start Import' and wait until the import process finishes.



The implementation of database backup and recovery protocols provides operational benefits for enterprise data systems. It saves data from unexpected attacks or human errors like unintentional

dropping or deleting important tables or records. Routine backups can decrease the work stoppage and save the restaurant from substantial financial losses because they ensure rapid disaster recovery without manual reconfiguration. Also, having a backup makes the system able to keep working without long stoppage, which helps to maintain the restaurant's reputation.

Food preparation and dispatch rely on real-time synchronization between couriers and incoming orders. If the database instance crashes during peak hours, recovery from a self-contained backup restores active delivery assignments and updates within `Delivery_Status` immediately, preventing stranded food deliveries and driver idle time.

7.2 Database maintenance in general

Database maintenance should be a scheduled and routine action to monitor the database and ensure its performance integrity and availability without having a work stoppage. In Jasmine restaurant's delivery management, the tables are going through many operations like inserting, updating and deleting, and with time these continuous operations will decrease the performance and make many errors in the database, if the system doesn't go through routine data maintenance. These errors will cause system crashes resulting in work stoppages and huge problems within the datasets, in order to resolve these problems, the restaurant would take a long time and spend a lot of money which could threaten the restaurant's financial stability.

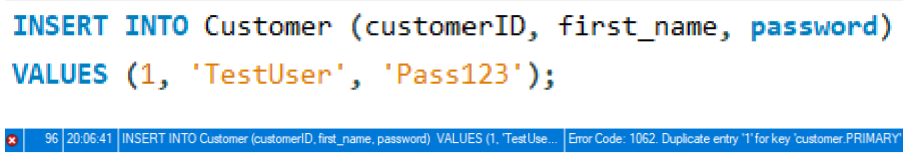
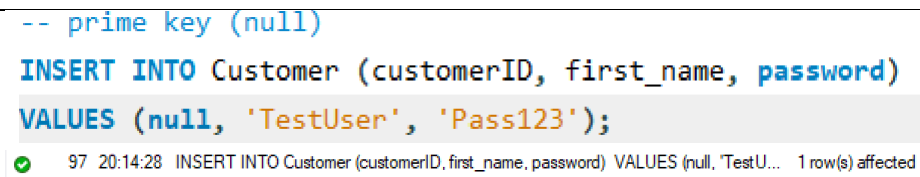
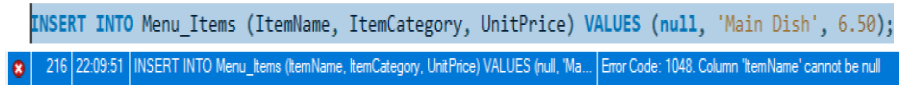
Methods of database maintenance that the restaurant can follow:

- 1) **Index Maintenance and Storage Defragmentation:**
Modifications that occur repetitively in active transactional tables lead to data page fragmentation and imbalance in indexing. The implementation of defragmentation routines will help organize storage space, reorganize clustered and secondary indexes, reduce disk input/output, and improve relational lookups.
- 2) **Structural Integrity Verification:**
This procedure involves running automated diagnostic routines to verify the consistency of internal storage pages, confirm that primary and foreign keys are connecting in a right way, and discover physical damage early, preventing unexpected transaction failures during production.
- 3) **Historical Data Archiving and Purging:**
Active operational tables should prioritize real-time workflow transactions. Scheduled maintenance migrates completed orders and old ones (which have been in the system for years without being used) into another storage, keeping production tables lean, reducing daily backup sizes, and speeding up daytime operational queries.

- 4) User Privilege Auditing and Security Hardening:
Periodic auditing of the access and privileges each user has to ensure strict following to the principle of least privilege. This includes deprecating obsolete employee profiles, enforcing scheduled password renewals, and auditing permission boundaries to mitigate insider and outsider security threats.

8 Testing

8.1 Data Validation

	Type	Description	screenshot
1	PK	The primary keys should be unique And there is already a customer with Id 1 so the database rises an error	 <pre>INSERT INTO Customer (customerID, first_name, password) VALUES (1, 'TestUser', 'Pass123');</pre>
2	PK	The primary keys should not be null However, when trying to enter a null value it did not raise an error because the primary key in customer has identified with Auto increment feature, so when trying to give a null value for the primary key it just give the next number in the order	 <pre>-- prime key (null) INSERT INTO Customer (customerID, first_name, password) VALUES (null, 'TestUser', 'Pass123');</pre>
3	PK	Here we are trying to insert a null value on a primary key which do not have auto increments feature as a result the data sets will refuse having the primary key as null and will raise an error	 <pre>INSERT INTO Menu_Items (ItemName, ItemCategory, UnitPrice) VALUES (null, 'Main Dish', 6.50);</pre>
4	FK	Here we are trying to connect the record in the	<pre>-- fk INSERT INTO Orders (OrderCustomerID, OrderPrice) VALUES (999, 15.00);</pre>

		orders table with non-existent record in Customer table . It raises errors because 999 is not an ID for any customer	 99 20:55:35 INSERT INTO Orders (OrderCustomerID, OrderPrice) VALUES (999, 15.00) Error Code: 1452. Cannot add or update a child row
5	FK	Here we are trying to delete a record from drivers table this driver is connected by a foreign key an attribute in orders table so because this the table of the drivers is defined as restrict it will not allow deletion while it was identified as set null it will allow the deletion and will set null value in the order driver ID also if it was defined as on delete cascades the driver's record and connected orders with it will get deleted	<pre>DELETE FROM drivers WHERE driverID = 1;</pre>  100 21:03:24 DELETE FROM drivers WHERE driverID = 1 Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fail...
6	FK	The foreign key The orders item is defined as on delete cascades which means if we want to delete an order the order will be deleted alongside with the order item record	<pre>INSERT INTO Orders (OrderID, OrderCustomerID, OrderPrice) VALUES (999, 1, 15.00); INSERT INTO Order_Items (ItemName, OrderID, Quantity) VALUES ('Mansaf Platter', 999, 1); DELETE FROM Orders WHERE OrderID = 999;</pre>  215 21:59:23 DELETE FROM Orders WHERE OrderID = 999 1 row(s) affected
7	Check	there is a constraint on Amount_paid attributes in payments_records table that's prevent to insert any negative value, so when trying to insert -3 in that attribute an error will be raised	<pre>insert into payment_records (OrderID , AmountPaid) values (5 , -3);</pre>  108 21:42:08 insert into payment_records (OrderID , AmountPaid) values (5 , -3) Error Code: 3819. Check constraint 'pos_AmountPaid' is violated.
8	Unique	There is unique constraint on customer phone number attributes so when you're trying to insert a phone number	<pre>insert into customer (first_name,phone ,password) values ('slamah' , '0791234567' , '3469jhgf');</pre>  111 21:48:36 insert into customer (first_name,phone ,password) values ('slamah' , '0791234567' , ... Error Code: 1062. Duplicate entry '0791234567' for key 'customer.phone'

		already exist in the customer table an error will be raised	
--	--	---	--

8.2 Output Validation

	Query Description	Screenshot	Result validation																														
1	In this command select has been used. And this query will select all the columns in the table only if the order price for the records is bigger than 10	<pre>-- using select with where select * from orders where OrderPrice > 10;</pre> <table><tr><th></th><th>OrderID</th><th>OrderCustomerID</th><th>OrderDriverID</th><th>PredictableTime</th><th>OrderPrice</th></tr><tr><td>▶</td><td>1</td><td>1</td><td>1</td><td>2026-09-01 13:45:00</td><td>17.00</td></tr><tr><td></td><td>3</td><td>2</td><td>2</td><td>2026-09-02 20:15:00</td><td>14.50</td></tr><tr><td></td><td>5</td><td>5</td><td>NULL</td><td>NULL</td><td>12.50</td></tr><tr><td></td><td>6</td><td>1</td><td>NULL</td><td>2026-09-05 17:30:00</td><td>12.50</td></tr></table>		OrderID	OrderCustomerID	OrderDriverID	PredictableTime	OrderPrice	▶	1	1	1	2026-09-01 13:45:00	17.00		3	2	2	2026-09-02 20:15:00	14.50		5	5	NULL	NULL	12.50		6	1	NULL	2026-09-05 17:30:00	12.50	A table appeared in the terminal window includes all the orders table columns alongside with all the records with order price bigger than 10 this command has done what was predicted from it successfully
	OrderID	OrderCustomerID	OrderDriverID	PredictableTime	OrderPrice																												
▶	1	1	1	2026-09-01 13:45:00	17.00																												
	3	2	2	2026-09-02 20:15:00	14.50																												
	5	5	NULL	NULL	12.50																												
	6	1	NULL	2026-09-05 17:30:00	12.50																												
2	This procedure is responsible for updating a price for certain item on the restaurant’s menu The user will enter the name of the item and the new price, and this procedure will update the price item. A call command has been used.	<pre>-- update the price drop procedure if exists p_UpdateMenuItemPrice; delimiter \$\$ create procedure p_UpdateMenuItemPrice (in p_ItemName VARCHAR(100) , in p_UnitPrice DECIMAL(6, 2)) begin update menu_items set UnitPrice = p_UnitPrice where ItemName = p_ItemName; end \$\$ delimiter ; CALL p_UpdateMenuItemPrice('Chicken Shawarma Meal', 6.75); SELECT ItemName, ItemCategory, UnitPrice FROM Menu_Items WHERE ItemName = 'Chicken Shawarma Meal';</pre> <table><tr><th></th><th>ItemName</th><th>ItemCategory</th><th>UnitPrice</th></tr><tr><td>▶</td><td>Chicken Shawarma Meal</td><td>Main Dish</td><td>6.75</td></tr></table>		ItemName	ItemCategory	UnitPrice	▶	Chicken Shawarma Meal	Main Dish	6.75	As we see from the table below the price of chicken Shawarma meal item in menu items table Has been updated to the price that has been determined when calling the procedure which is 6.75.																						
	ItemName	ItemCategory	UnitPrice																														
▶	Chicken Shawarma Meal	Main Dish	6.75																														
3	Here a view has been created to show the order bill details including the order ID the item name and category the	<pre>DROP VIEW IF EXISTS vw_Order_Bill_Details; CREATE VIEW vw_Order_Bill_Details AS SELECT o.OrderID, i.ItemName , m.ItemCategory, i.Quantity, m.UnitPrice from orders o join order_items i on o.OrderID = i.OrderID join menu_items m on m.ItemName = i.ItemName; select * FROM vw_Order_Bill_Details;</pre>	After querying the order bill details view, all wanted columns have been shown in one table alongside with their values.																														

	quantity and the item unit price in order to view all these columns in one table together	<table><tr><th>OrderID</th><th>ItemName</th><th>ItemCategory</th><th>Quantity</th><th>UnitPrice</th></tr><tr><td>1</td><td>Arabic Salad</td><td>Side Dish</td><td>1</td><td>2.50</td></tr><tr><td>1</td><td>Fresh Orange Juice</td><td>Drink</td><td>NULL</td><td>2.00</td></tr><tr><td>1</td><td>Mansaf Platter</td><td>Main Dish</td><td>1</td><td>12.50</td></tr><tr><td>1</td><td>Mineral Water</td><td>Drink</td><td>2</td><td>0.75</td></tr><tr><td>2</td><td>Chicken Shawarma Meal</td><td>Main Dish</td><td>1</td><td>6.75</td></tr><tr><td>2</td><td>French Fries</td><td>Side Dish</td><td>1</td><td>1.50</td></tr><tr><td>2</td><td>Fresh Orange Juice</td><td>Drink</td><td>1</td><td>2.00</td></tr><tr><td>6</td><td>Arabic Salad</td><td>Side Dish</td><td>1</td><td>2.50</td></tr></table>	OrderID	ItemName	ItemCategory	Quantity	UnitPrice	1	Arabic Salad	Side Dish	1	2.50	1	Fresh Orange Juice	Drink	NULL	2.00	1	Mansaf Platter	Main Dish	1	12.50	1	Mineral Water	Drink	2	0.75	2	Chicken Shawarma Meal	Main Dish	1	6.75	2	French Fries	Side Dish	1	1.50	2	Fresh Orange Juice	Drink	1	2.00	6	Arabic Salad	Side Dish	1	2.50	
OrderID	ItemName	ItemCategory	Quantity	UnitPrice																																												
1	Arabic Salad	Side Dish	1	2.50																																												
1	Fresh Orange Juice	Drink	NULL	2.00																																												
1	Mansaf Platter	Main Dish	1	12.50																																												
1	Mineral Water	Drink	2	0.75																																												
2	Chicken Shawarma Meal	Main Dish	1	6.75																																												
2	French Fries	Side Dish	1	1.50																																												
2	Fresh Orange Juice	Drink	1	2.00																																												
6	Arabic Salad	Side Dish	1	2.50																																												
4	Using select commands to display payments methods alongside with the number of each payment method.	<pre>select PaymentMethod , count(PaymentMethod) from payment_records group by PaymentMethod;</pre> <table><thead><tr><th>PaymentMethod</th><th>count(PaymentMethod)</th></tr></thead><tbody><tr><td>Cash</td><td>3</td></tr><tr><td>CliQ</td><td>2</td></tr><tr><td>Credit Card</td><td>1</td></tr></tbody></table>	PaymentMethod	count(PaymentMethod)	Cash	3	CliQ	2	Credit Card	1	A table containing payment methods that customers have used before with a number usage for every method. The sum of counts methods column equal 6 which is similar to the number of records in payment records table																																					
PaymentMethod	count(PaymentMethod)																																															
Cash	3																																															
CliQ	2																																															
Credit Card	1																																															

8.3 Security Validation

privileges given:

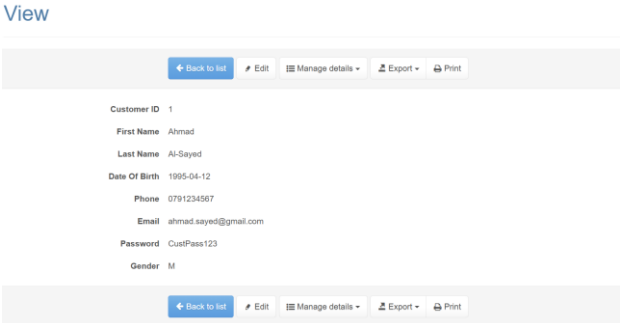
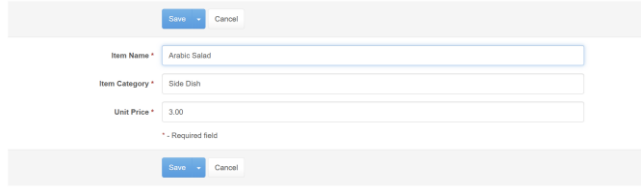
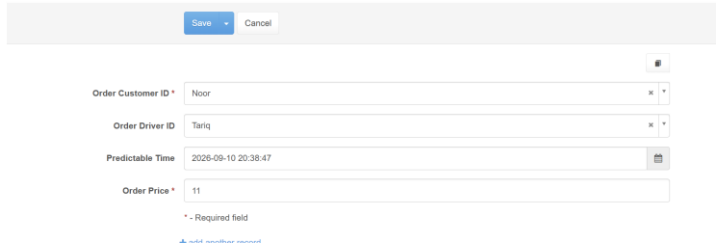
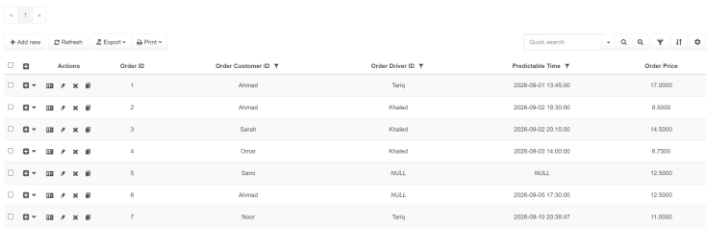
	Username	Description of privilege/no privilege	Screenshot (query + result)																																			
1	driver	Each driver can access the delivery_status table to see the order status and he can update the table too.	<pre>select * from delivery_status; update delivery_status set DeliveryStatusName = 'out for delivery' where StatusID = 5;</pre> <table><tr><th>StatusID</th><th>OrderID</th><th>DeliveryStatusName</th><th>UpdatedAt</th><th>Notes</th></tr><tr><td>1</td><td>1</td><td>delivered</td><td>2026-09-01 13:55:00</td><td>Delivered to customer doorstep successfully.</td></tr><tr><td>2</td><td>2</td><td>out for delivery</td><td>2026-09-02 19:20:00</td><td>Driver is near the destination.</td></tr><tr><td>3</td><td>3</td><td>out for delivery</td><td>2026-09-06 18:34:11</td><td>NULL</td></tr><tr><td>4</td><td>4</td><td>cancelled</td><td>2026-09-03 13:40:00</td><td>Cancelled upon customer phone request.</td></tr><tr><td>5</td><td>5</td><td>out for delivery</td><td>2026-09-03 14:15:00</td><td>NULL</td></tr><tr><td>6</td><td>6</td><td>preparing</td><td>2026-09-06 18:34:11</td><td>NULL</td></tr></table>	StatusID	OrderID	DeliveryStatusName	UpdatedAt	Notes	1	1	delivered	2026-09-01 13:55:00	Delivered to customer doorstep successfully.	2	2	out for delivery	2026-09-02 19:20:00	Driver is near the destination.	3	3	out for delivery	2026-09-06 18:34:11	NULL	4	4	cancelled	2026-09-03 13:40:00	Cancelled upon customer phone request.	5	5	out for delivery	2026-09-03 14:15:00	NULL	6	6	preparing	2026-09-06 18:34:11	NULL
StatusID	OrderID	DeliveryStatusName	UpdatedAt	Notes																																		
1	1	delivered	2026-09-01 13:55:00	Delivered to customer doorstep successfully.																																		
2	2	out for delivery	2026-09-02 19:20:00	Driver is near the destination.																																		
3	3	out for delivery	2026-09-06 18:34:11	NULL																																		
4	4	cancelled	2026-09-03 13:40:00	Cancelled upon customer phone request.																																		
5	5	out for delivery	2026-09-03 14:15:00	NULL																																		
6	6	preparing	2026-09-06 18:34:11	NULL																																		
2	Driver	The driver can also access to vw_active_deliveries Which allows the driver to see useful information about the orders and who ordered them that will helps them in their job.	<pre>select * from vw_active_deliveries;</pre> <table><tr><th>driverID</th><th>first name driver</th><th>first name customer</th><th>customer phone</th><th>OrderID</th><th>OrderPrice</th><th>PaymentMethod</th><th>AmountPaid</th></tr><tr><td>2</td><td>Khaled</td><td>Ahmad</td><td>0791234567</td><td>2</td><td>8.50</td><td>ClIQ</td><td>8.50</td></tr><tr><td>2</td><td>Khaled</td><td>Sarah</td><td>0782345678</td><td>3</td><td>14.50</td><td>Credit Card</td><td>14.50</td></tr></table>	driverID	first name driver	first name customer	customer phone	OrderID	OrderPrice	PaymentMethod	AmountPaid	2	Khaled	Ahmad	0791234567	2	8.50	ClIQ	8.50	2	Khaled	Sarah	0782345678	3	14.50	Credit Card	14.50											
driverID	first name driver	first name customer	customer phone	OrderID	OrderPrice	PaymentMethod	AmountPaid																															
2	Khaled	Ahmad	0791234567	2	8.50	ClIQ	8.50																															
2	Khaled	Sarah	0782345678	3	14.50	Credit Card	14.50																															
3	Customer	Customer can access to menu_items table to see	<pre>select * from menu_items;</pre>																																			

		the items in the menu their price and other details about them.	<table><tr><th>ItemName</th><th>ItemCategory</th><th>UnitPrice</th></tr><tr><td>▶ Arabic Salad</td><td>Side Dish</td><td>2.50</td></tr><tr><td>Chicken Shawarma Meal</td><td>Main Dish</td><td>6.75</td></tr><tr><td>French Fries</td><td>Side Dish</td><td>1.50</td></tr><tr><td>Fresh Orange Juice</td><td>Drink</td><td>2.00</td></tr><tr><td>Knafeh Plate</td><td>Dessert</td><td>3.50</td></tr><tr><td>Mansaf Platter</td><td>Main Dish</td><td>12.50</td></tr><tr><td>Mineral Water</td><td>Drink</td><td>0.75</td></tr></table>	ItemName	ItemCategory	UnitPrice	▶ Arabic Salad	Side Dish	2.50	Chicken Shawarma Meal	Main Dish	6.75	French Fries	Side Dish	1.50	Fresh Orange Juice	Drink	2.00	Knafeh Plate	Dessert	3.50	Mansaf Platter	Main Dish	12.50	Mineral Water	Drink	0.75	
ItemName	ItemCategory	UnitPrice																										
▶ Arabic Salad	Side Dish	2.50																										
Chicken Shawarma Meal	Main Dish	6.75																										
French Fries	Side Dish	1.50																										
Fresh Orange Juice	Drink	2.00																										
Knafeh Plate	Dessert	3.50																										
Mansaf Platter	Main Dish	12.50																										
Mineral Water	Drink	0.75																										
4	Admin	Admin can access to vw_admin_order_summary That allows the admin to see customer orders and who deliver them, which is a very important privilege to help the admin track orders..	select * from vw_admin_order_summary; <table><tr><th>OrderID</th><th>customerID</th><th>customer first name</th><th>driverID</th><th>driver first name</th></tr><tr><td>1</td><td>1</td><td>Ahmad</td><td>1</td><td>Tariq</td></tr><tr><td>2</td><td>1</td><td>Ahmad</td><td>2</td><td>Khaled</td></tr><tr><td>3</td><td>2</td><td>Sarah</td><td>2</td><td>Khaled</td></tr><tr><td>4</td><td>3</td><td>Omar</td><td>2</td><td>Khaled</td></tr></table>	OrderID	customerID	customer first name	driverID	driver first name	1	1	Ahmad	1	Tariq	2	1	Ahmad	2	Khaled	3	2	Sarah	2	Khaled	4	3	Omar	2	Khaled
OrderID	customerID	customer first name	driverID	driver first name																								
1	1	Ahmad	1	Tariq																								
2	1	Ahmad	2	Khaled																								
3	2	Sarah	2	Khaled																								
4	3	Omar	2	Khaled																								

privileges not given:

	Username	Description of privilege/no privilege	Screenshot (query + result)
1	Driver	Driver can access to delivery_status table just to view or to update but he cannot insert a new record in the table	<pre>insert into delivery_status (OrderID , DeliveryStatusName) values (1, 'out for delivery');</pre> 9 18:56:21 insert into delivery_status (OrderID , DeliveryStatusName) values (1, 'out for delivery') Error Code: 1142. INSERT command denied to user 'driver'@'localhost' for table 'del...
2	Driver	Driver does not have access to any table except delivery_status so he can not view customer table records	<pre>-- driver can not access to Customer tabel select * from Customer;</pre> 10 19:00:30 select * from Customer LIMIT 0, 10000 Error Code: 1142. SELECT command denied to user 'driver'@'localhost' for table 'cu...
3	Customer	customer can access to menu_items table just to view but he cannot delete record from the table	<pre>delete from jasmine_restaurant.menu_items where UnitPrice = 3 ;</pre> 3 19:04:57 delete from jasmine_restaurant.menu_items where UnitPrice = 3 Error Code: 1142. DELETE command denied to user 'customer'@'localhost' for table '...
4	Customer	Customer does not have access to the orders table, so he cannot update, modify, or view the table records.	<pre>update Orders set OrderDriverID = 2 where OrderID = 2;</pre> 4 19:10:14 update Orders set OrderDriverID = 2 where OrderID = 2 Error Code: 1142. UPDATE command denied to user 'customer'@'localhost' for table '...

8.4 GUI Validation:

	Description	screenshot
1	After clicking on the view icon on customer record in the customer page, customer's information appears on the screen	
2	After clicking on the pencil icon in the Menu item page an edit page will appear and will allow us to update the record information. Here Unit Price has been updated to 3.00JD after being 2.50JD	
3	After clicking on Add new button an Add new page appears that let us add new order. And require the entering the new order information	
4	After clicking on the X icon a warning message appears on the screen asks the user to confirm the deletion process and if the user clicks yes the record will be deleted. The order with Id = 7 has been deleted	

Orders						
+ Add new ↻ Refresh 📄 Export + 🖨 Print + Quick search ⌵ 🔍 🗲 ⌨ ⚙						
☐	☐	Actions	Order ID	Order Customer ID ▼	Order Driver ID ▼	Predictable Time ▼ Order Price
☐	☒	☐ ☐ ☐ ☐ ☐ ☐	1	Ammad	Tariq	2025-09-01 13:45:00 17.0000
☐	☒	☐ ☐ ☐ ☐ ☐ ☐	2	Ammad	Khaled	2025-09-02 19:30:00 8.5000
☐	☒	☐ ☐ ☐ ☐ ☐ ☐	3	Sarah	Khaled	2025-09-02 20:15:00 14.5000
☐	☒	☐ ☐ ☐ ☐ ☐ ☐	4	Omar	Khaled	2025-09-03 14:00:00 6.7500
☐	☒	☐ ☐ ☐ ☐ ☐ ☐	5	Sami	NULL	NULL 12.5000
☐	☒	☐ ☐ ☐ ☐ ☐ ☐	6	Ammad	NULL	2025-09-06 17:30:00 12.5000

8.5 Assess whether meaningful data has been extracted

Meaningful data has been inserted into Jasmine Restaurant Management Delivery dataset. This data simulates the end-to-end operational workflow, and it ensures a referential integrity in all the entities in the dataset, including the integrity of the relations between the tables.

For example, customer_address has been separated from customer main table into another table called customer_address because customer address is a complex attribute that contains in it two simple attributes and can have multi-values. In the dataset, Ahmed customer who has ID=1 relates to multi locations, house location type and work location type successfully. This connection has been successfully implemented by applying a foreign key in customer_address refers to customer primary key.

An example of many-to-many relationships in the dataset is the relation between orders table and Menu_Items table. Each order can contain multi-items or one item, while each item can be in zero order or multi-orders. For example, Arabic salad is included in multiple orders, like the order with ID=1 and the order with ID=6. On the other hand, the order with ID=1 is containing multiple items like Arabic salad and fresh juice. This process requires moving order_items from orders table and creating its own table, foreign key has been used alongside to connect these two tables.

An example of a one-to-one relationship is the relation between Orders table and Payment record. Each order can have one and only one payment record, and each payment record can relate to one and only one order.

Procedures and views have been designed in order to serve the operational process efficiently. For each one of the users (admin, driver, or customers) have views and procedures that help them while using the delivery management system. For example, VW_Admin_Order_Summary view is responsible for collecting the data from drivers table, orders table, and customer table, which allows the admins to track the order's process easily in a professional way. Join command has been used in order to display these different attributes from multi-tables together in one table at the screen.

Another example of useful views in the dataset is VW_ACTIVE_DELIVERIES view. This view displays to the drivers an important information that relates to the delivery tasks. It displays

customers' phone numbers and price details while isolating the previous records (delivered or cancelled orders) and customers' passwords to maintain the dataset safety. As the previous view, join command has been used, but now with where condition command to prevent displaying canceled or delivered orders.

One of the automated procedures in the dataset is P_NEW_CUSTOMER_order, which automates entering attributes in multi-tables like orders table, order items table, etc..., which prevents orphan records and decreases the errors while inserting. It uses six input parameters with insert commands.

Another procedure is P_ASSIGNDRIVERTOOrder. This procedure assigns drivers and updates the delivery status alongside with updated_at attributes using Now() function. It uses two input parameters with update commands.

Jasmine restaurant delivery management system replaces manual paper records alongside with tables that are saved in papers which reduces the threats of losing the data or duplicating the bills. This reflects the restaurant's overall performance, quality, accuracy, and safety, keeping the restaurant reputation safe and helps the restaurant extending its branches, while organizing the financial sector in more professional way. It also, helps detecting any errors or threats in a real-time situation.

8.6 Assess the effectiveness of testing

The next essential stage after building the dataset and writing the SQL code is testing. The testing stage aims to check if the dataset which was built following the physical schema that has been drawn before alongside its constraints and user's privileges that have been decided, while ensuring the user interface is working in professional and effective a way. There are several steps in testing stage. The first step is checking the efficiency and the integrity and enforcement of constraints built into the dataset. on the dataset, such as the primary keys cannot be null and they should be unique. In testing, we check if the dataset accepts null values or duplicated primary keys.

Also in this step we have tested the integrity of the foreign keys and other constraints like check and unique. These tests confirm that the database prevents inserting wrong values.

The second step is ensuring that the commands, views, and aggregate functions are working correctly alongside with stored procedures. This ensures that the internal logic for the database is accurate and predictable, and that is what Jasmine Restaurant Delivery Management System is on.

The third step is verifying the implementation of the least privilege principle through creating different users (admin, driver, and customer). Here, we ensure that every user can access the operations that he needs while preventing the user from other operations that he does not have

the privilege for, with warning message if he tried to access them. For example, driver users shouldn't reach to the customer table that includes customer passwords. So, the driver user does not have this privilege, and with testing, we ensure that least privilege principle is working in a predictable way.

Finally, the user interface was tested to ensure that it is user-friendly, and it reflects the database in the right way, including its operations like viewing, inserting, deleting, or updating. The testing stage is very important and efficient for ensuring the database integrity. By applying these tests, we have ensured that Jasmine Restaurant Delivery Management System is working in an effective way following all the constraints and security privileges.

9 Evaluation of database solution

9.1 Effectiveness of the database solution based on user and system requirement

The main reason behind implementing this management system is to stop relying on paper records and unstructured spreadsheets, replacing them with a secure, automated, and scheduled relational architecture that organizes the table, gives detailed records, especially in the financial field, and increases the system integrity and security. And that's what we have built in this project.

At the beginning of this project, database requirements have been set. One of them is customer requirements. In this database management system, many requirements have been achieved, for the customer example: sign up requirement. Customer can sign up on the system using their full name, phone number, etc. Also, users can see the items of their orders. In addition, customers can track the orders by the delivery status table, displaying each dish information such as price and category, and being able to see the order price and each item unit price individually.

In the other side, there are few requirements were not achieved or were postponed from the core relational prototype. Like forget password recovery mechanism and customer rating with comments. Because forget password requirement recovery needs accessing to external protocols like SMTP and other web interfaces for checking a verification code and these things located outside the internal SQL range. Also, customer rating and comments have been postponed keeping the SQL management system simple and focus on the main operations and requirements to finish.

A new feature that has been added to the customer requirement and wasn't mentioned in the first version of data requirement is displaying the order bill details `vw_Order_Bill_Details` which let the customer see all the important financial information about the order in easy way. This feature has been added because of its great importance by making the database more user friendly and trying to display all the information that a customer needs in one place.

For driver's requirements, a lot of features have been implemented like sign in, tracking order, showing the predictable time that the customer will get the order, and the delivery status. Being able to see preparing orders or out for delivery orders. However, as the customer requirement, forget password button hasn't been established because of the reason that has been mentioned before. Also, calculating the distance between the customer and the restaurant requirement hasn't been applied. Because it needs access for the real-time location of the customers and access for the geographic maps and locations, which have been postponed focusing on database efficiency.

For the admin, the database has implemented many requirements like management, restaurant profile, and operational information. The admin can manage all the database, customers, and drivers' profiles alongside different information. Also, admin can access different private information about the restaurant, profit, and the cost of the orders. He can also manage the pricing operation by updating menu items price. Finally, admin can assign drivers to orders.

A new feature that has been added to the admin is tracking the order details with customer details and driver details all in one table using view. This helps the admin track the order, who ordered it, and who will deliver it

For non-functional requirements (system requirements the achieved requirements include fast, secure because the database follows least privilege principle, easy to use, the database and the user interface is very user-friendly, good capacity, the database able to contain many customers, drivers, and admins, backup, a backup has been export of the SQL, and the database is easy to update.

In conclusion, the developed database solution effectively fulfills the operational objectives of Jasmine Restaurant by bridging user needs with a resilient relational structure. The platform replaces error-prone manual operations with consistent transactional integrity, procedural automation, and strict access controls, establishing a scalable foundation capable of supporting the restaurant's ongoing operational expansion.

9.2 Suggested improvements

specialized users

One of the best enhancements that can be added to the current database is specialized users. In the current database, there are three different types of users: customer, driver, and admin. In most of delivery management systems, the number of user types is more than three. We can add a restaurant accountant, database developer and the restaurant manager which means dividing admin user into three different users and define the privileges for each one of them to follow the least privilege principle.

Encrypting passwords

One of the greatest enhancements that can increase the security for the database is encrypting the passwords. In the database system, password attribute is defined as varchar (255). This allocated storage space (VARCHAR(255)) can be utilized to store hashed passwords securely.

More attributes

Add more attributes for the tables to make the database more accurate. For example, in `menu_items`, adding warning notes for the items that can cause allergic reactions in certain people.

Adding a new entity

Adding a new entity, such as inventory entity, that tracks ingredients availability in the restaurants, kitchen, or storage, and connect this entity with `menu_items`.

Partitioning

Transferring old records like old completed orders into different storage space to increase the scalability of the database management system and performance.

9.3 Evaluation based on improvements needed

Implementing these enhancements is not just a simple adding operation, but every modification on the database should be taken after deep analysis of performance, accuracy, security, financial demands, and restaurant abilities.

specialized users

Dividing user accounts into three different users this enforced of implementing least privilege principle because the financial accountant will be just able to access the payment records and other bills rather than make the accountant reach the data sets by the admin user account. However, dividing a user account into other users' accounts may cause a bigger management source demand to track and control the privileges in the data set. Their team maintenance will take more time with more users.

Encrypting passwords

Cryptographic key derivation functions transform clear password texts into cryptographic hashes, which provides strong protection against attacks. Even if the attacker succeeds in reaching the customer's passwords, he will not be able to read them or use these passwords, which protects the customer's safety. However, cryptographic key derivation functions create a big load on the CPU because it needs a lot of long calculations, which increase the latency and decrease the performance of the dataset. During peak operational orders hours, these encrypting passwords may generate a big pressure on the CPU, this CPU will start tracking too much time, which will affect the database availability.

More attributes

Expanding the `Menu_Items` entity to include domain-restricted allergen flags directly addresses commercial food safety compliance and customer welfare. Storing critical medical warnings directly alongside dish descriptions eliminates operational reliance on customer inquiries, protecting the restaurant from health liabilities.

While adding non-key descriptive columns introduces negligible storage footprints, the primary operational challenge lies in data maintenance and user-interface recalibration. Every new menu classification requires immediate validation and updates from kitchen supervisors, introducing risks of stale or inaccurate allergen entries if inventory supplier brands change. Moreover, the client-facing GUI must be refactored to parse dynamic allergen tags, requiring additional front-end validation logic and query payload bandwidth.

If the restaurant was bringing ingredients that do not cause allergic reaction and did not put allergic alerts in the `menu_items` and the restaurant decided to buy this ingredient from another source without checking if this in new ingredients cause allergic reaction and did not update its `menu_items`, that might cost the restaurant its reputation and a big financial losses.

Adding a new entity

Integrating a standalone Inventory module connected to `Menu_Items` via a `Recipe_Ingredients` association establishes a closed-loop supply chain. Programmatically decrementing raw ingredient levels upon order placement automates procurement schedules and eliminates overselling by dynamically marking dishes unavailable when ingredients are depleted.

This enhancement may increase the schema complexity. The one order will not just deal with `orders` table and `order_items` table, but it will also start dealing with a third table. Also, tracking the usage of ingredients will increase database developing time and the length of testing stage.

Partitioning

Applying range partitioning to timestamp columns—such as partitioning orders and order items by season or month—resolves the inevitable data volume expansion associated with projected annual growth. Furthermore, offloading older, archived data to separate storage media maintains sub-second index query speeds and optimizes cache utilization for active orders, ensuring that current transactions are processed without the latency of full table scans. Disaster recovery procedures become more complex; restoring a specific corrupted partition requires specialized administrative steps rather than a simple restore from a unified backup file, thereby raising the level of technical expertise required of the database maintenance team.

In conclusion, implementing these enhancements may enhance the effectiveness of the database or its security. However, taking enhancement decisions needs an accurate balance between operational earnings and time developing financial demands and management effort to maintain this more complex database.

Moreover, these enhancements raise the financial demands because they need to increase the number of CPUs, servers, and storage space to deal with cryptographic and complex schemas.

So, the best implementation approach in Jasmine Restaurant is gradually, in stages—focusing first on tightening security and maintaining performance and subsequently integrating advanced inventory systems in parallel with the restaurant's operational expansion.

10 User Documentation

10.1 System Overview

Introduction:

Electronic database for Jasmine Restaurant delivery management system. Its target is to abandon the paper documents and organize the delivery system. It connects the customer, driver and admins with user friendly program.

Key Features & Capabilities:

Customer:

- Enable customers to Sign in
- Display items menu details (name, price, category)
- Generate and display bills details
- Tracking the order
- Allow customers to store multiple delivery addresses (e.g., Home, Work) for fast checkout

Driver:

- Viewing orders, their price and customer name and his phone number
- Viewing the delivery status predictable time for the orders
- Update the delivery status.

Admin:

- Control the data console
- Manage difference tables.
- Track the orders, their price, the customer who ordered them, and the driver who will deliver them.

Basic Architecture & User Roles:

Customers: sign in, sign up, display the menu, submitting orders and tracking the delivering process.

Drivers: display orders with ('preparing' or 'out for delivery orders') delivery status , display customers phone numbers and updating order delivery status.

Administrators: supervise and control the data, accounts and operations.

Types of Data Managed

Customer: Menu items: dish names, price, category, order bills, his own profile information

Driver: Order & Payment data: the items it contains and predictable time to arrive and the payment method, some attributes in the customer table(like the name and phone number)

Admin: Full access to manage and supervise all operational data across the database, including customer and driver accounts, menu catalog pricing, financial record, and menu items.

System Limitations:

- Requires an active internet connection to submit orders and receive live status updates.
- Real-time updates depend on server availability and uninterrupted network connectivity.

10.2 Using the system

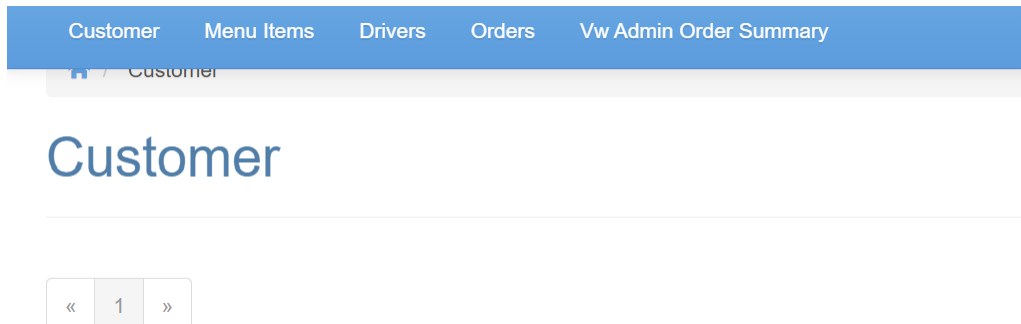
Admin user:

Insert: adding a record in a column.

Admin can insert records in any table in the database

Example insert record in table:

Step 1: chose customer tab from the upper panel



Step 2: click on '+Add new' button

« 1 »

+ Add new

Refresh

Export

Print

Quick search

Q

Q

Y

IT

Q

☐	Actions	Customer ID	First Name	Last Name	Date Of Birth	Phone	Email	Password	Gender
☐	▼ 🔍 🗑️ 🔄 🖨️	1	Ahmad	Al-Sayed	1995-04-12	0791234567	ahmad.sayed@gmail.com	CustPass123	M
☐	▼ 🔍 🗑️ 🔄 🖨️	2	Sarah	Mansour	1998-09-23	0782345678	sarah.mansour@yahoo.com	CustPass123	F
☐	▼ 🔍 🗑️ 🔄 🖨️	3	Omar	Khatib	2001-01-15	0773456789	omar.khatib@outlook.com	CustPass123	M
☐	▼ 🔍 🗑️ 🔄 🖨️	4	Noor	Haddad	1992-11-05	0794567890	noor.haddad@gmail.com	CustPass123	F
☐	▼ 🔍 🗑️ 🔄 🖨️	5	Sami	solami	NULL	0794567860	Sami.solami@gmail.com	CustPass123	M
☐	▼ 🔍 🗑️ 🔄 🖨️	6	Nora	NULL	1992-11-05	NULL	NULL	CustPass123	NULL

Step 3: write customer information

First Name * tala

Last Name toman

Date Of Birth 2026-09-15 

Phone 0777676790

Email tala@gimal.com

Password * 123456789

Gender F

* - Required field

Save Cancel

Step 4: click on 'save'

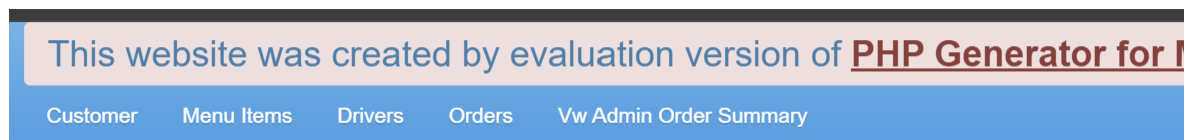
☐			7	tala	toman	2026-09-15	0777676790	tala@gimal.com	123456789	F
--------------------------	---	---	---	------	-------	------------	------------	----------------	-----------	---

update: modifying a record in a column by changing one or more than value in its attribute.

Admin can update records in any table in the database

Example update record in table Drivers

Step 1: go to driver tab



Step 2: click on the pencil icon in the record we want to update

Drivers

« 1 »

+ Add new

↻ Refresh

📄 Export

🖨 Print

Quick see

☐	+	Actions	Driver ID	First Name	Last Name	Date Of Birth ▼	Phone	Email
☐	+	📄 ✎ ✖ 🗑	1	Tariq	Qasim	1994-03-10	0795551122	tariq.driver@jasmine.com
☐	+	📄 ✎ ✖ 🗑	2	Khaled	Najjar	1996-07-18	0786662233	khaled.driver@jasmine.com
☐	+	📄 ✎ ✖ 🗑	3	Zaid	Othman	1999-12-01	0777773344	zaid.driver@jasmine.com
☐	+	📄 ✎ ✖ 🗑	4	Mustafa	NULL	NULL	0798889900	mustafa.driver@jasmine.com
☐	+	📄 ✎ ✖ 🗑	5	Hassan	Ali	1997-05-20	NULL	NULL

« 1 »

Step 3: update the desired attribute for example date of birth

First Name *

Hassan

Last Name

Ali

Date Of Birth

1997-05-20

📅

Phone

Email

Password *

DriverPass123

Gender

* - Required field

Save

Cancel

Save

Cancel

First Name *

Hassan

Last Name

Ali

Date Of Birth

2000-08-15

📅

Phone

Email

Password *

DriverPass123

Gender

* - Required field

Save

Cancel

Step 4: click ‘save’

☐				5	Hassan	Ali	2000-08-15	NULL	NULL	DriverPass123	NULL
--------------------------	--	--	--	---	--------	-----	------------	------	------	---------------	------

Delete Operation remove a record from the data
(Example: Deleting a Canceled Order):

Step 1: Click on the Orders tab from the top navigation bar to open the orders table.

CustomerMenu ItemsDriversOrdersVw Admin Order Summary

Orders

Orders

«1»

Step 2: Locate the record to delete (e.g., Order ID = 6) and click on the 'X' (Delete) icon in the Actions column.

Orders

«1»

+ Add new

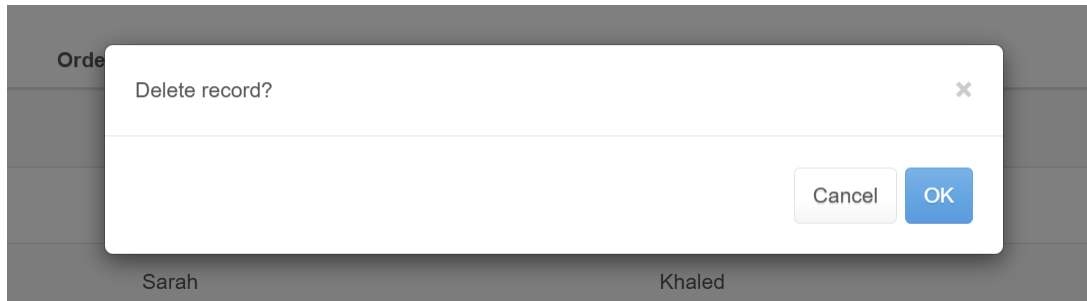
Refresh

Export

Print

☐	+	Actions	Order ID	Order Customer ID
☐	+ ▼		1	Ahmad
☐	+ ▼		2	Ahmad
☐	+ ▼		3	Sarah
☐	+ ▼		4	Omar
☐	+ ▼		5	Sami
☐	+ ▼		6	Ahmad

Step 3: A confirmation popup dialog will appear asking to confirm the deletion. Click 'OK' / 'Yes' to execute the deletion.



Orders

« 1 »

+ Add new

Refresh

Export

Print

☐	Actions	Order ID	Order Customer ID
☐		1	Ahmad
☐		2	Ahmad
☐		3	Sarah
☐		4	Omar
☐		5	Sami

Display a view:

- Select the 'Vw Admin Order Summary' tab from the top navigation menu.

This website was created by evaluation version of [PHP Generator for MySQL Professional](#).

Customer Menu Items Drivers Orders Vw Admin Order Summary

Vw Admin Order Summary

« 1 »

Refresh

Export

Print

Quick search

☐	Order ID	Customer ID	Customer First Name	Driver ID	Driver First Name
☐	4	3	Omar	2	Khaled
☐	3	2	Sarah	2	Khaled
☐	2	1	Ahmad	2	Khaled
☐	1	1	Ahmad	1	Tariq

« 1 »

Viewing Individual Record Details: displaying the record details.

Step 1: In any table (such as the Customer table), locate the desired row.

« 1 »

+ Add new

↻ Refresh

📄 Export ▾

🖨 Print ▾

Quick search

▾ 🔍 🔍 🔍 ⚙

☐	Actions	Customer ID	First Name	Last Name	Date Of Birth ▾	Phone	Email	Password	Gender
☐	▾ 🆔 ✎ ✖ 🖨	1	Ahmad	Al-Sayed	1995-04-12	0791234567	ahmad.sayed@gmail.com	CustPass123	M
☐	▾ 🆔 ✎ ✖ 🖨	2	Sarah	Mansour	1998-09-23	0782345678	sarah.mansour@yahoo.com	CustPass123	F
☐	▾ 🆔 ✎ ✖ 🖨	3	Omar	Khatib	2001-01-15	0773456789	omar.khatib@outlook.com	CustPass123	M
☐	▾ 🆔 ✎ ✖ 🖨	4	Noor	Haddad	1992-11-05	0794567890	noor.haddad@gmail.com	CustPass123	F
☐	▾ 🆔 ✎ ✖ 🖨	5	Sami	solami	NULL	0794567860	Sami.solami@gmail.com	CustPass123	M
☐	▾ 🆔 ✎ ✖ 🖨	6	Nora	NULL	1992-11-05	NULL	NULL	CustPass123	NULL

Step 2: Click on the View icon (ID card icon) located in the Actions column.



Step 3: A dedicated breakdown screen opens, displaying all attributes of the selected entity in a clean profile format.

View

← Back to list

✎ Edit

📄 Manage details ▾

📄 Export ▾

🖨 Print

Driver ID 1

First Name Tariq

Last Name Qasim

Date Of Birth 1994-03-10

Phone 0795551122

Email tariq.driver@jasmine.com

Password DriverPass123

Gender M

← Back to list

✎ Edit

📄 Manage details ▾

📄 Export ▾

🖨 Print

Customer Role (End-User Operations)

Customer interface is designed as a user-friendly self-service portal for registered customers allowing browsing menu items, placing new orders, and tracking their orders as well: **Browsing Menu Catalog:** Registered customers can browse available categories of food (Main Dish, Side Dish, Drink, Dessert) after they sign into their accounts. The menu catalog shows names of dishes, their categories, and unit prices. **Placing New Orders:** To place a new order, customers have to select their dishes, indicate quantity, choose a delivery destination and a payment method (Cash, Credit Card, or CliQ). Once the order is submitted, a new order will be created and associated with selected meal items, the procedure also initializes status as "preparing" and creates a payment entry as well. **Live Order Tracking & Billing Verification:** After an order is submitted, a customer can monitor the order's status in real-time using a customer tracking view (vw_customer). This view shows the actual delivery status, the estimated delivery time and even courier contact information. Additionally, customers can use vw_Order_Bill_Details view to see the list of ordered dishes and itemized prices of each order's item.

Customers can not insert any record in a table directly, but they can view tables by clicking on table name tab and then clicking on view icon.

Driver Role (Courier Operations)

The driver interface creates a simple portal that is specifically designed to help the drivers view the information they need, reach their destination, and follow their deliveries: **Viewing Active Tasks:** The driver can see the delivery list via vw_Active_Deliveries interface. This interface filters out all the completed and canceled tasks, leaving only those which are either under 'preparing or "out for delivery". **Seeing Order Information and Customer Contacts:** In every task allocated to the courier, he will see the information about the order including order ID, customer name, phone number and the mode of payment for the order (card/CliQ payment or cash on delivery). **Getting Unassigned Deliveries:** If an order is ready but there is no assigned driver to make the delivery, then the driver acquires the task using the p_AssignDriverToOrder procedure, where the driver ID is linked to the order. **Milestone Tracking During Delivery:** The driver enters the Delivery_Status table while achieving milestones during delivery (like changing the status from " preparing " to "out for delivery" and then completing the order with the label "delivered").

The driver can update delivery status stable by going to delivery status tap click on it thin click on the pencil icon in the record he wants to update finally he can click on save.

For displaying we will repeat all these steps but instead clicking the updating icon we will click on view icon instead.

10.3 Frequently asked questions

Question 1: Why the system produces an error message "Cannot delete or update a parent row: a foreign key constraint fails"?

Cause: The relational table "Order" is being used as a Parent row in the foreign key constraint in Delivery_Status or Payment_Records tables which restricts deletion. Troubleshooting steps:

Do not delete the active orders or those that are in transit from the grid. In case an order needs to be deleted, it should be canceled from the status attribute of the Delivery_Status table by changing its value to "cancelled". Deletion is allowed in case of test records only.

Question 2: Why does the registration form reject the creation of a new customer or driver with an error message "Duplicate entry for key 'phone'"?

Cause: There is a UNIQUE integrity constraint defined for contact attributes such as Phone and Email in the Customer and Drivers entities to avoid duplicate identities and ensure proper routing for dispatches. Troubleshooting steps:

Make sure that the mobile phone number or email address is not associated with any of the existing customers or drivers. Edit the existing record in lieu of the creation of the duplicate record.

Question 3: Why does a delivery driver not see delivered or canceled orders in his active dispatch view?

Cause: The courier application routes the deliveries via the relational view vw_Active_Deliveries which has a server-side filter WHERE DeliveryStatusName IN ('preparing', 'out for delivery'). This ensures that only active routes will be shown to optimize the screen space.

Troubleshooting steps: Delivered and canceled orders are removed from the dispatch list automatically. To investigate shift records and canceled invoice records, please contact your shift administrator who has access to the audit relational view vw_Admin_Order_Summary.

Question 4: What happens in case of submission of the invalid payment methods and negative unit prices via the interface?

Cause: The database has the domain verification constraint at the schema level such as CHECK (UnitPrice >= 0), CHECK (OrderPrice >= 0), and CHECK (PaymentMethod IN ('Cash', 'Credit Card', 'CliQ')).

Troubleshooting steps: Please make sure that dish prices and order prices are non-negative numeric values. Please use only the transaction gateway services allowed in the payment drop-down list: 'Cash', 'Credit Card', and 'CliQ'. Otherwise, the default value is taken as 'Cash' if the payment service was not selected at all.

10.4 Contact Information

For technical support, reporting operational anomalies, or requesting user manual updates regarding the Jasmine Restaurant Delivery Management System, users can contact the dedicated IT and support channels below:

- **Technical Support Email:** support@jasmine-restaurant.com
- **IT Operations & Dispatch Hotline:** +962 6 570 2890 / +962 7 9532 0462
- **Operating Hours:**
 - Technical Support Team: Sunday to Thursday, 8:00 AM – 10:00 PM
 - Emergency Dispatch & Server Monitoring: Available 24/7
- **Physical Office:** Jasmine Restaurant Headquarters – Operations & IT Support Division, Amman, Jordan

Online Help Resources & Community Channels

- **Staff & Driver Support Forum:**
- (<https://community.jasmine-restaurant.com/dispatch-forum>)
(an internal platform for couriers and dispatch managers to exchange best practices, report routing issues, and suggest UI improvements).
- **Customer Feedback & Ticketing System:** Accessible via the mobile application profile menu under **Help & Support** to submit real-time inquiries and track ticket resolution.

References:

- Connolly, T. and Begg, C. (2015) *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th edn. Boston: Pearson Education.
- Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems*. 7th edn. Boston: Pearson.
- Silberschatz, A., Korth, H.F. and Sudarshan, S. (2020) *Database System Concepts*. 7th edn. New York: McGraw-Hill.
- Bell, C. (2018) *Expert MySQL: Database Administration and Tuning*. 2nd edn. New York: Apress.
- Coronel, C. and Morris, S. (2018) *Database Systems: Design, Implementation, & Management*. 13th edn. Boston: Cengage Learning.
- DuBois, P. (2014) *MySQL*. 5th edn. Indianapolis: Addison-Wesley Professional.
- Gertz, M. and Jajodia, S. (2008) *Handbook of Database Security: Applications and Trends*. New York: Springer.
- Hoffer, J.A., Ramesh, V. and Topi, H. (2019) *Modern Database Management*. 13th edn. New York: Pearson.
- Pratt, P.J. and Last, M.Z. (2015) *A Guide to SQL*. 9th edn. Boston: Cengage Learning.